Roma Tre

Information Visualization

Drawing Hierarchical Graphs

Part I

M. Patrignani

 copyright ©2026 m.patrignani

Copyright notice

- All the pages/slides in this presentation, including but not limited to, images, photos, animations, videos, sounds, music, and text (hereby referred to as “material”) are protected by copyright
- This material, with the exception of some multimedia elements licensed by other organizations, is property of the authors and/or organizations appearing in the first slide
- This material, or its parts, can be reproduced and used for didactical purposes within universities and schools, provided that this happens for non-profit purposes
- Any other use is prohibited, unless explicitly authorized by the authors on the basis of an explicit agreement
- This copyright notice must always be redistributed together with the material, or its portions

 copyright ©2026 m.patrignani

Credits





- Walter Didimo and Giuseppe Liotta (Perugia University)
  - provided me with their slides on hierarchical drawings
  - http://www.diei.unipg.it/

 copyright ©2026 m.patrignani

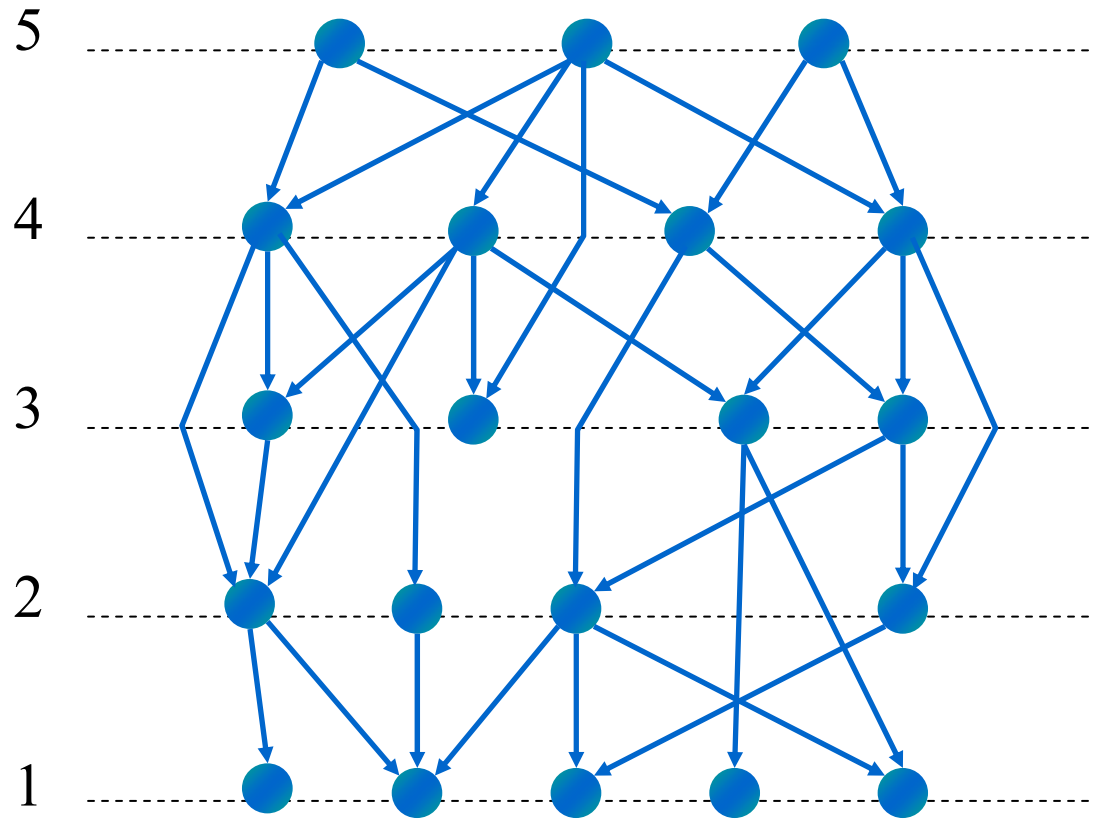
Summary

- The Sugiyama approach
- Step 1: Cycle removal
  - simple heuristics
  - greedy cycle removal
- Step 2: Level assignment
  - the Longest-path method
  - the Coffman-Graham method

 copyright ©2026 m.patrignani

Hierarchical drawings

- In a hierarchical drawing of a directed graph
  - nodes are drawn on a collection of horizontal lines (*levels* or *layers*)
  - edges are represented with *polylines* (*polygonal chains*)



 copyright ©2026 m.patrignani

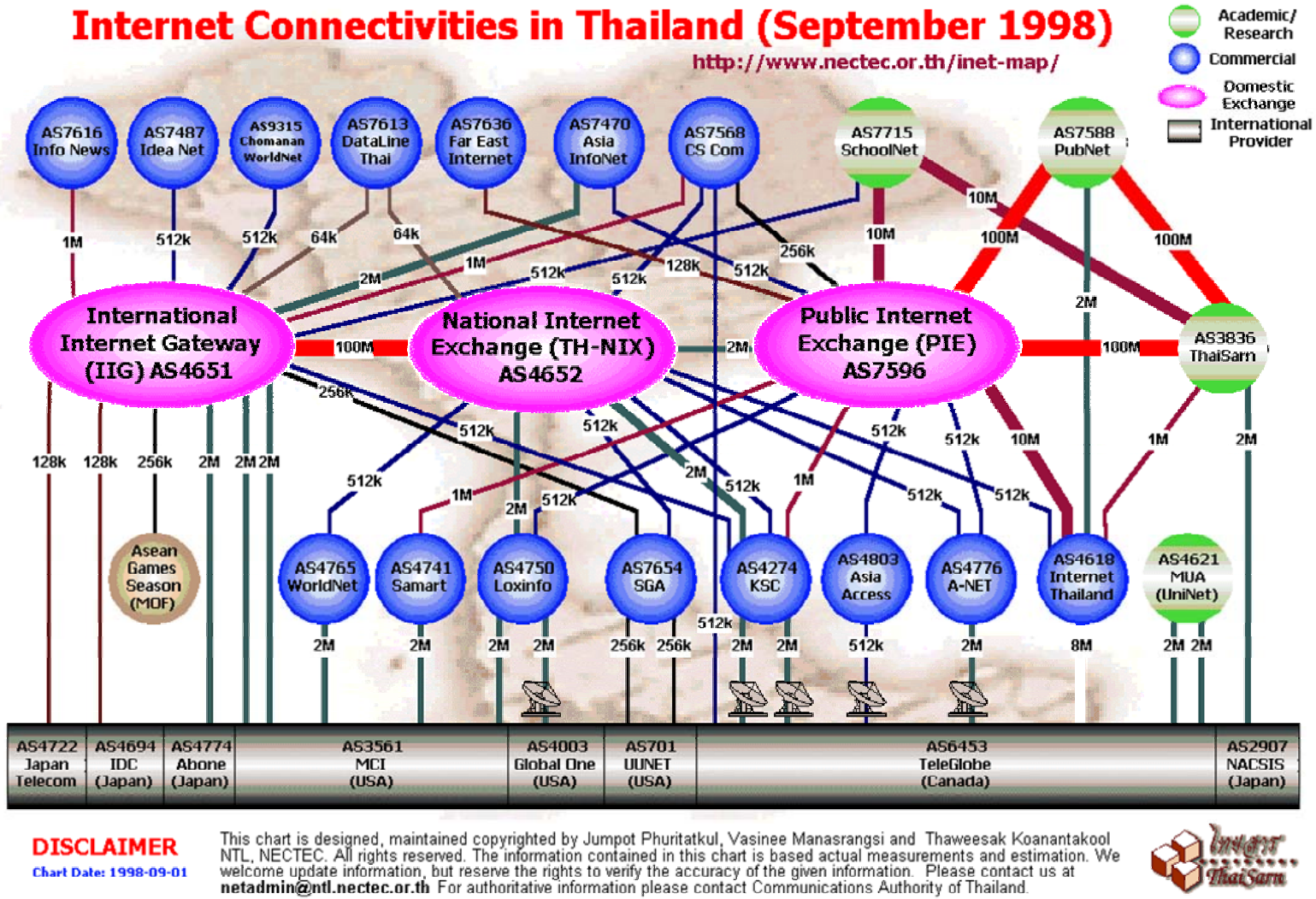
Application examples

- Hierarchical drawings are used to represent precedence relationships
- Applications include
  - computer network visualization
  - web maps visualization
  - visualization of biologic maps
  - social networks visualization
  - …

 copyright ©2026 m.patrignani

Computer networks visualization

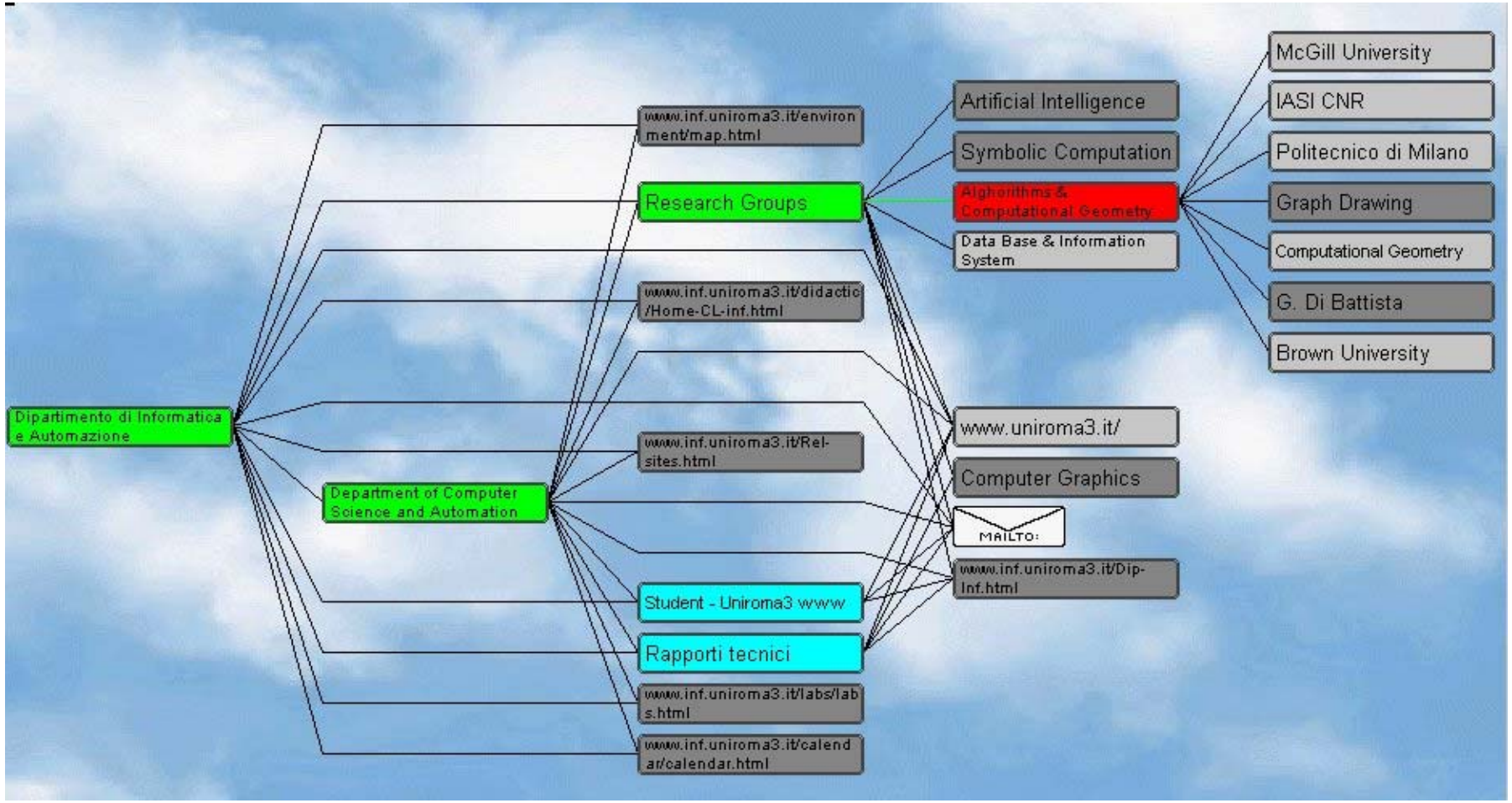
- Thailand's Internet Connectivity in Sep 1998
  - by J. Phuritatkul and T. Koanantakoolof NECTEC



 copyright ©2026 m.patrignani

Web maps visualization

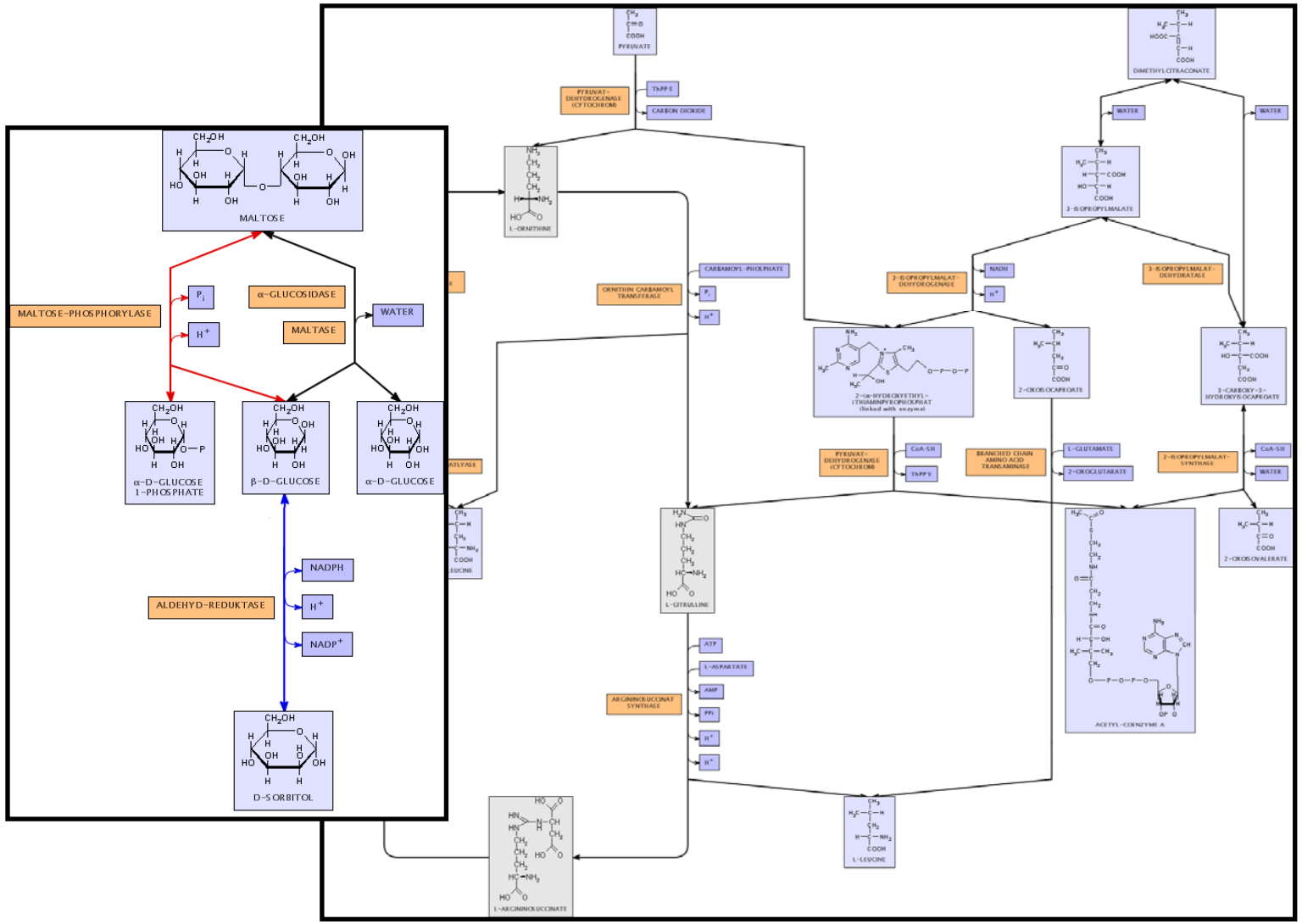
- Ptolomaeus [Di Battista et al 1998]



 copyright ©2026 m.patrignani

Pathways visualization

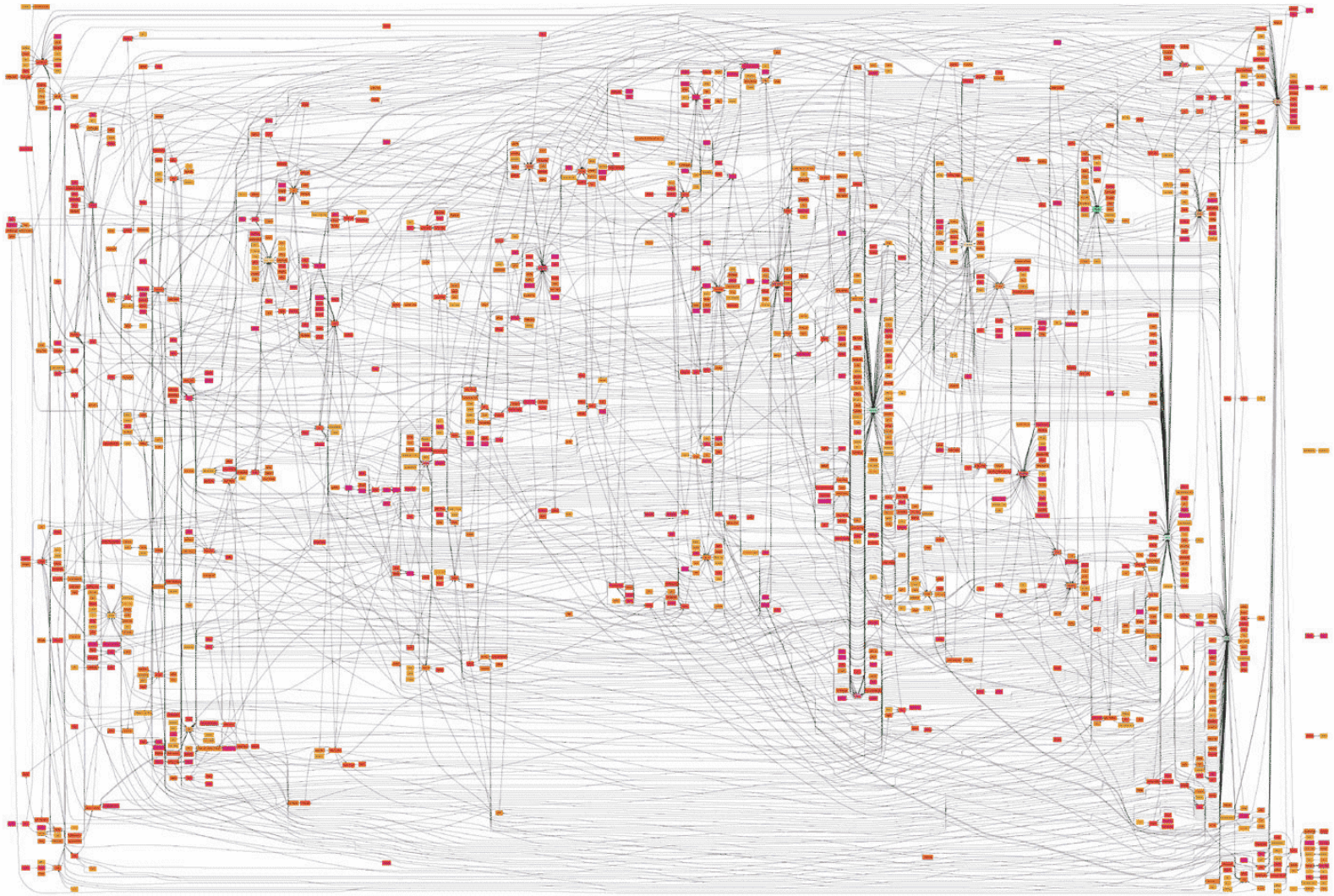
- Biochemical pathways
  - [Forster et al 2002]



 copyright ©2026 m.patrignani

Social networks visualization

- Social relationships inside a textual virtual space
  - Cobot system



 copyright ©2026 m.patrignani

Layered drawings: aesthetics

- Downwardness
  - as many as possible edges have to be oriented downward
- Reduced crossings
  - crossings impact readability
- Reduced number of the bends
- Reduced size (area)
- Reduced edge length

 copyright ©2026 m.patrignani

The layered methodology

- First introduced by [Sugiyama *et al.*, 81]
- Consists of four distinct steps
  - several alternative algorithms and heuristics are nowadays available for each step
- Widely spread
  - simple
  - intuitive
  - can draw any type of directed graph

 copyright ©2026 m.patrignani

Kozo Sugiyama 杉山公造 (1945-2011)

- Ph.D. in 1974
- Worked at Fujitsu until 1997
- Professor at the Japan Advanced Institute of Science and Technology (JAIST)
- Main contributions
  - layered graph drawing
  - concept of “mental map”
  - clustered graph drawing
  - magnetic forces in spring embedding algorithms
  - ...



copyright ©2026 m.patrignani

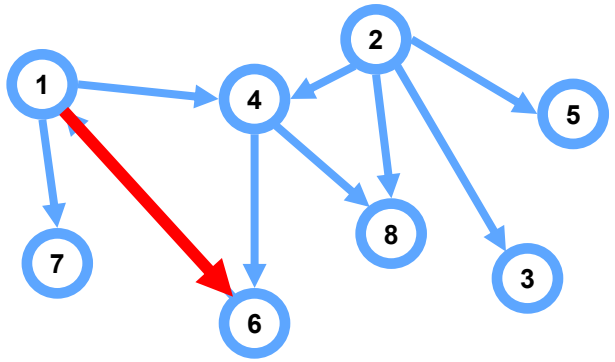
The four steps of the methodology

- Cycle removal
  - temporarily reverts the direction of some edges in such a way to have an acyclic input graph
  - this step is not needed if the input graph has no directed cycle
- Level assignment
  - assigns nodes to levels (hence determining their y-coordinate)
  - the graph is changed in such a way that each edge connects nodes on adjacent levels
    - this is obtained by introducing dummy nodes
- Crossing reduction
  - sorts the nodes of each level in such a way to reduce the number of edge crossings
- X-coordinate assignment
  - assigns an x-coordinate (hence a placement) to each node
  - dummy nodes are replaced with bends

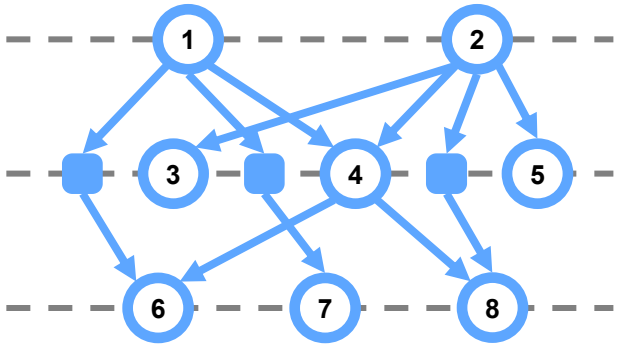
 copyright ©2026 m.patrignani

Example

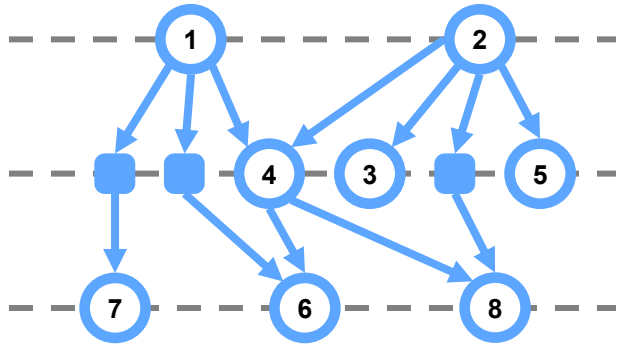
Cycle removal



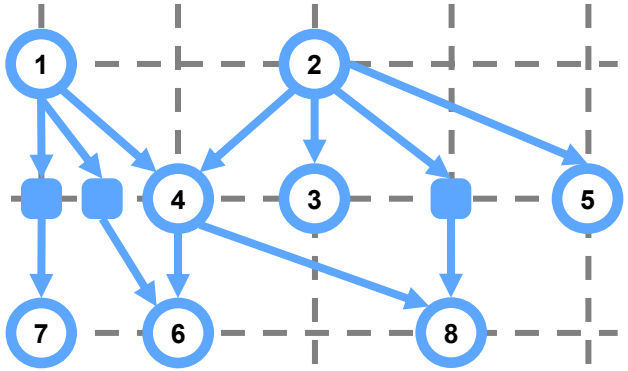
Level assignment



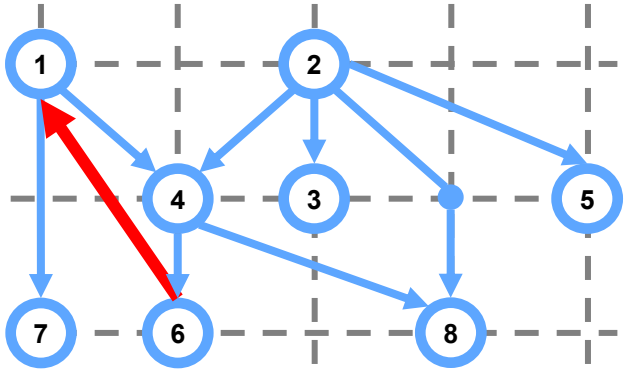
Crossing reduction



x-Coordinate assignment

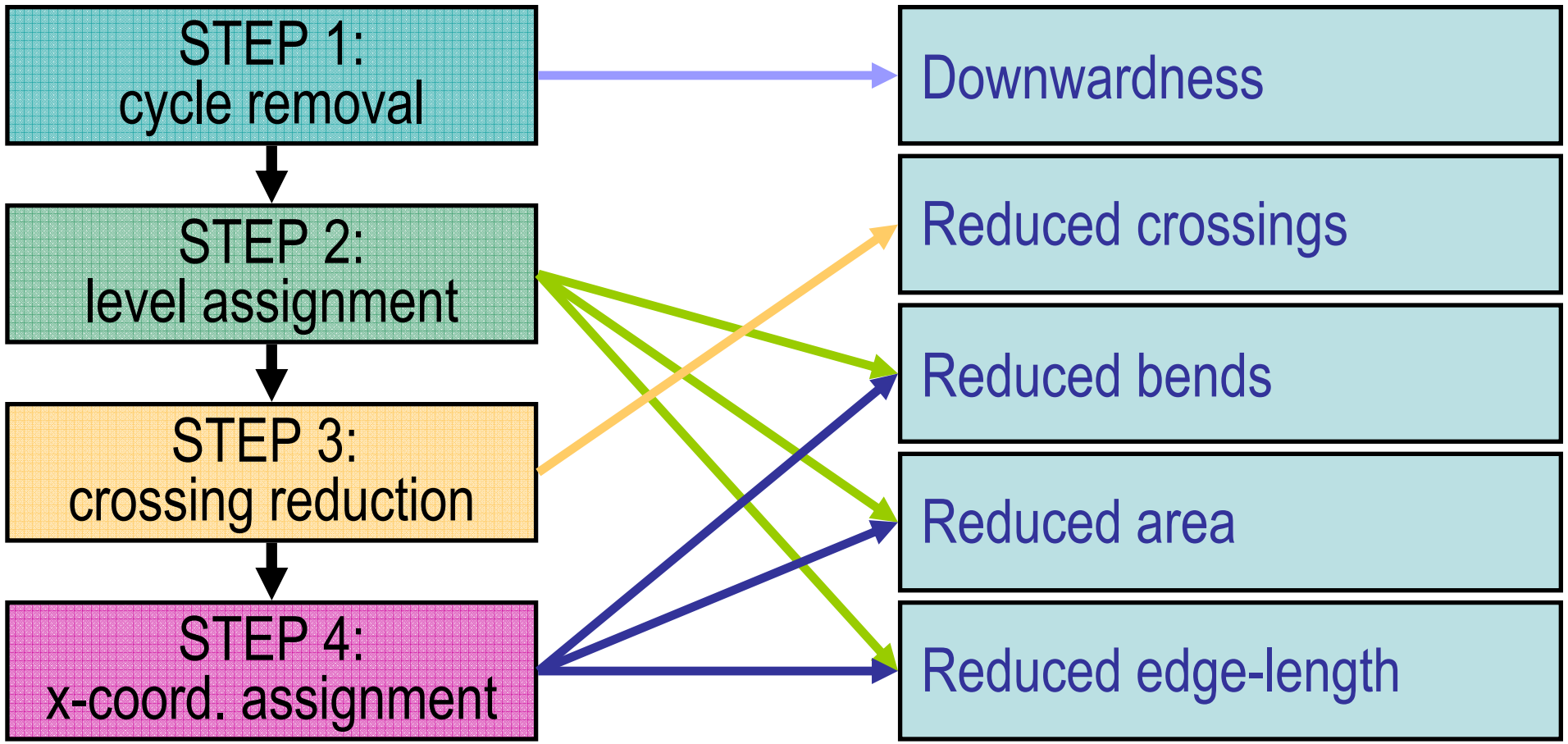


Final drawing



 copyright ©2026 m.patrignani

Algorithm steps and aesthetics



 copyright ©2026 m.patrignani

Step 1: Cycle removal

 copyright ©2026 m.patrignani

Problem statement

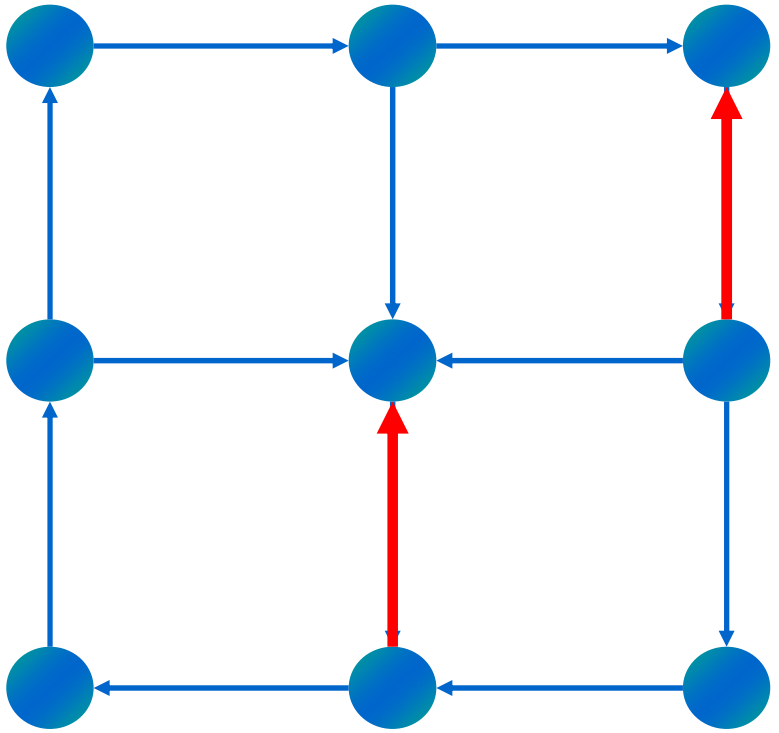
Input	A directed graph G
Output	A directed acyclic graph (DAG) G’
Target	Reverse the direction of a small subset of the edges of G

- If G=(V,E) is a directed graph and R is a set of edges, $G_{rev(R)}$ is the graph obtained by reversing the edges in R
- We would like to reverse a set of edges R so that $G_{rev(R)}$ is acyclic and |R| is minimum

 copyright ©2026 m.patrignani

Feedback set

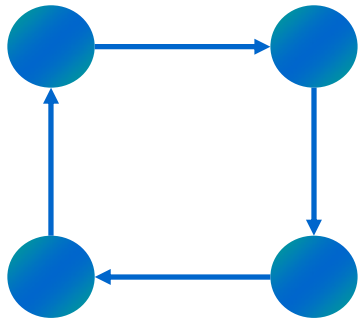
- A set R of edges of a directed graph G = (V,E) is a *feedback set* if $G_{rev(R)}$ is acyclic

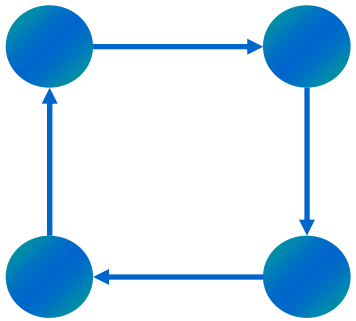


 copyright ©2026 m.patrignani

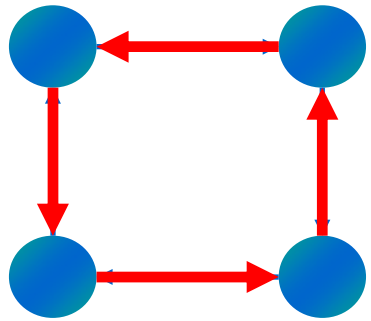
Feedback arc set

- A *feedback arc set* is a set of edges whose removal makes the graph acyclic
- Feedback set and feedback arc set are related but different concepts
  - removing all edges of a cycle makes it trivially acyclic
  - reversing all edges of a cycle only changes its direction





feedback arc set



not a feedback set

copyright ©2026 m.patrignani

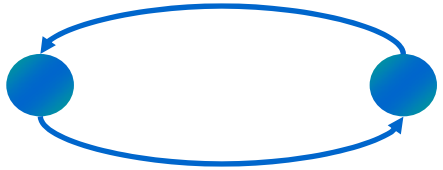
Feedback set and feedback arc set

- However, it can be shown that
  - a minimum size feedback set is a minimum size feedback arc set and vice versa
  - hence, the problem of determining a feedback set of minimum size for a given directed graph is equivalent to the well known feedback arc set problem
    - [Grinberg & Dambit, 66]
- Unfortunately, minimum feedback arc set is NP-complete [Karp 72]
  - for this reason several heuristics have been proposed

 copyright ©2026 m.patrignani

Feedback (arc) set lowerbound

- A bi-cycle of a directed graph G consists of and edge (u,v) and an edge (v,u)



  - directed graphs with many bi-cycles do not admit small feedback sets
    - each feedback set must contain at least one of the two edges of each bi-cycle
- There are instances of the feedback (arc) set problem whose solution has $\Omega(|E|/2)$ edges
  - no algorithm can do better than $|E|/2$

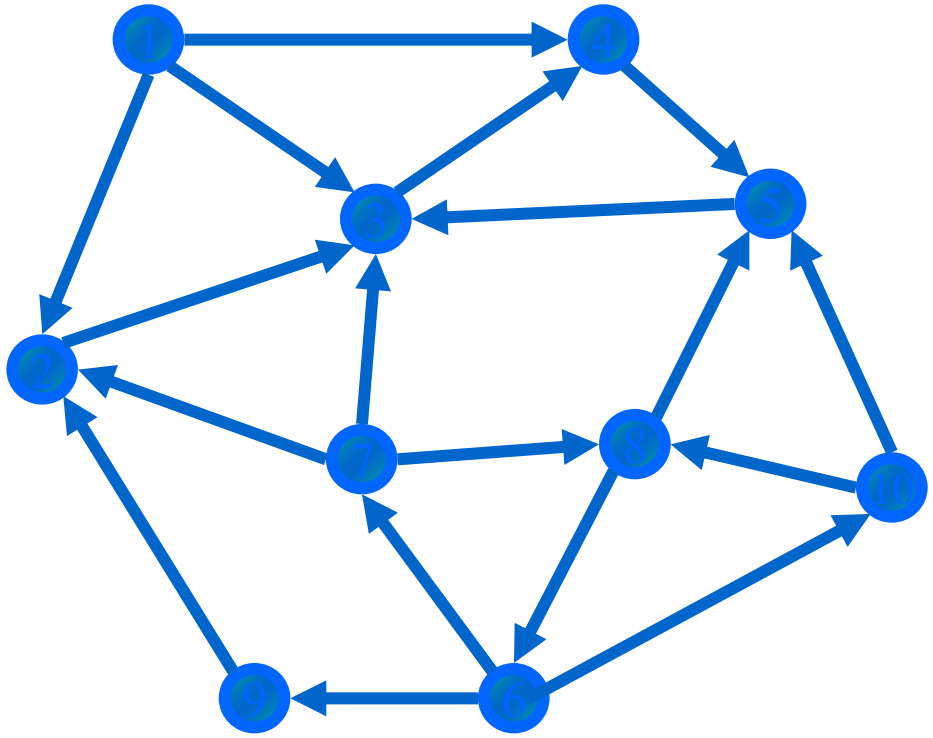
 copyright ©2026 m.patrignani

A simple heuristic

- A simple algorithm to find a feedback set R of a directed graph G consists of performing a series of DFS (Depth First Search) traversals of G
  - DFS visits are launched on non-explored nodes until the whole graph is explored
  - each node that is visited is given an increasing number
  - all edges to nodes already visited in the current DFS and with lower number are added to R

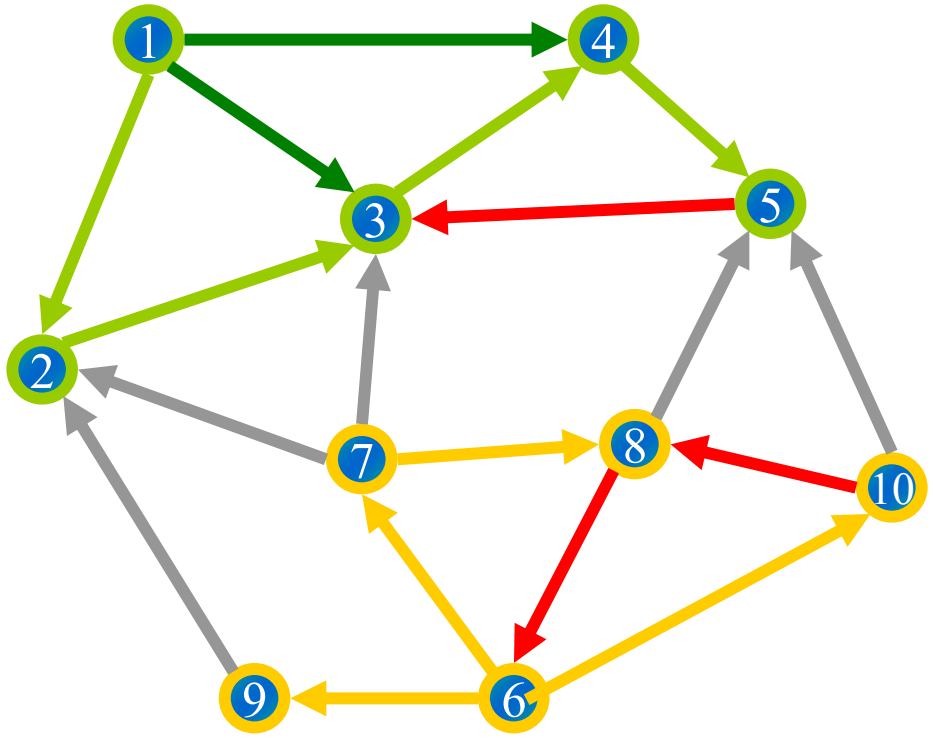
 copyright ©2026 m.patrignani

A simple heuristic: example



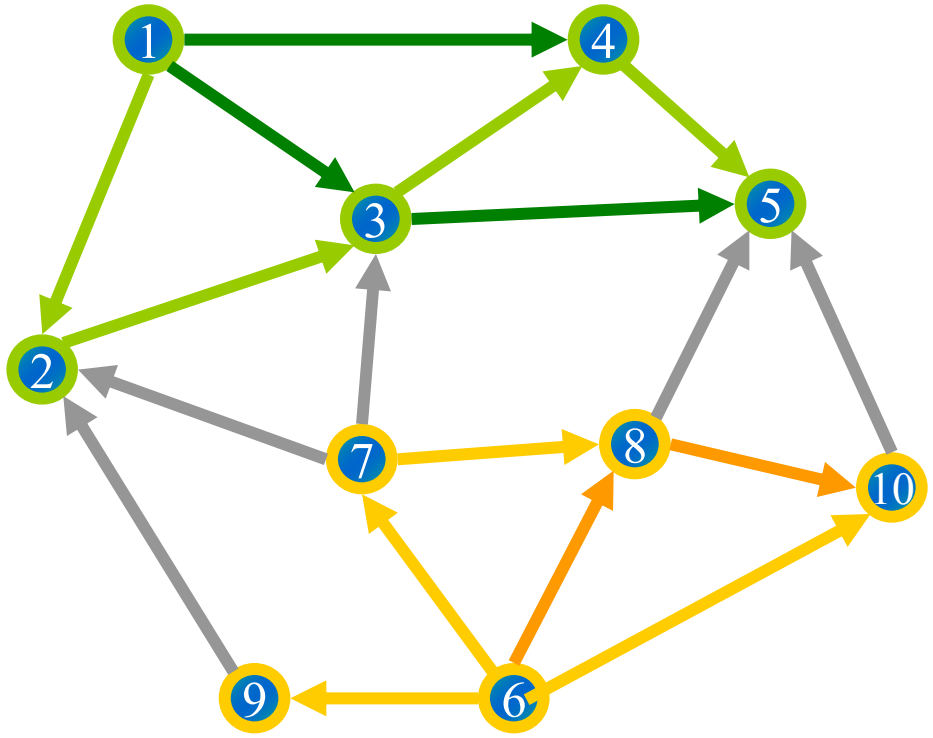
 copyright ©2026 m.patrignani

A simple heuristic: example



 copyright ©2026 m.patrignani

A simple heuristic: example



 copyright ©2026 m.patrignani

A simple heuristic: correctness

- Suppose to reverse all edges in R
  - these were edges going from higher-numbered to lower-numbered nodes
  - now they go from lower to higher-numbered nodes
- Edges among nodes of the same DFS visit
  - all go from lower-numbered to higher-numbered nodes
  - they cannot form directed cycles
- Edges among nodes of different DFS's
  - all go from nodes of later DFS's to nodes of previous DFS's
  - they cannot close directed cycles

 copyright ©2026 m.patrignani

A simple heuristic: analysis

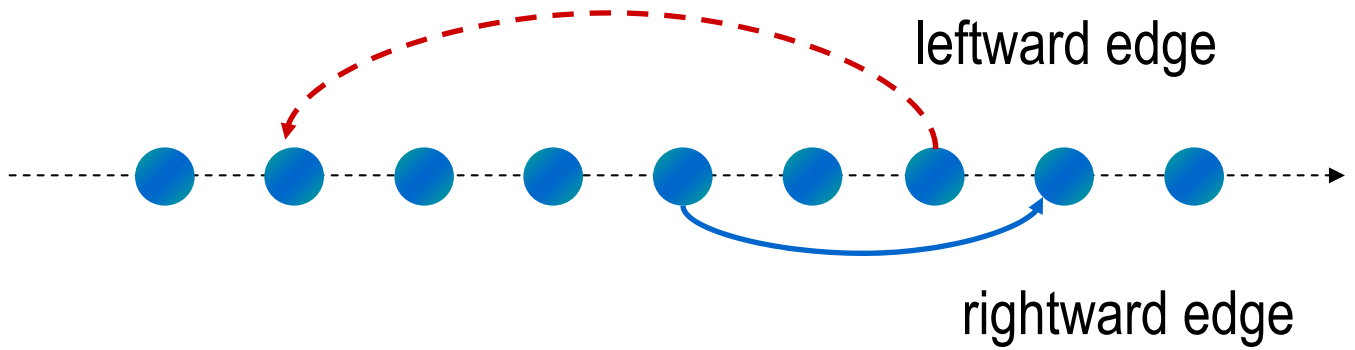
- Size of the feedback set R
  - we assume G is connected
  - if a single DFS is performed then the forward edges are a spanning tree with |V|-1 edges
  - if more DFS's are performed each DFS gives a tree and at least one edge links nodes of one DFS to some nodes of the previous DFS's
    - hence, once again we have |V|-1 "good" edges
  - the size of R is

$$|R| = |E|-(|V|-1) = |E|-|V|+1$$

 copyright ©2026 m.patrignani

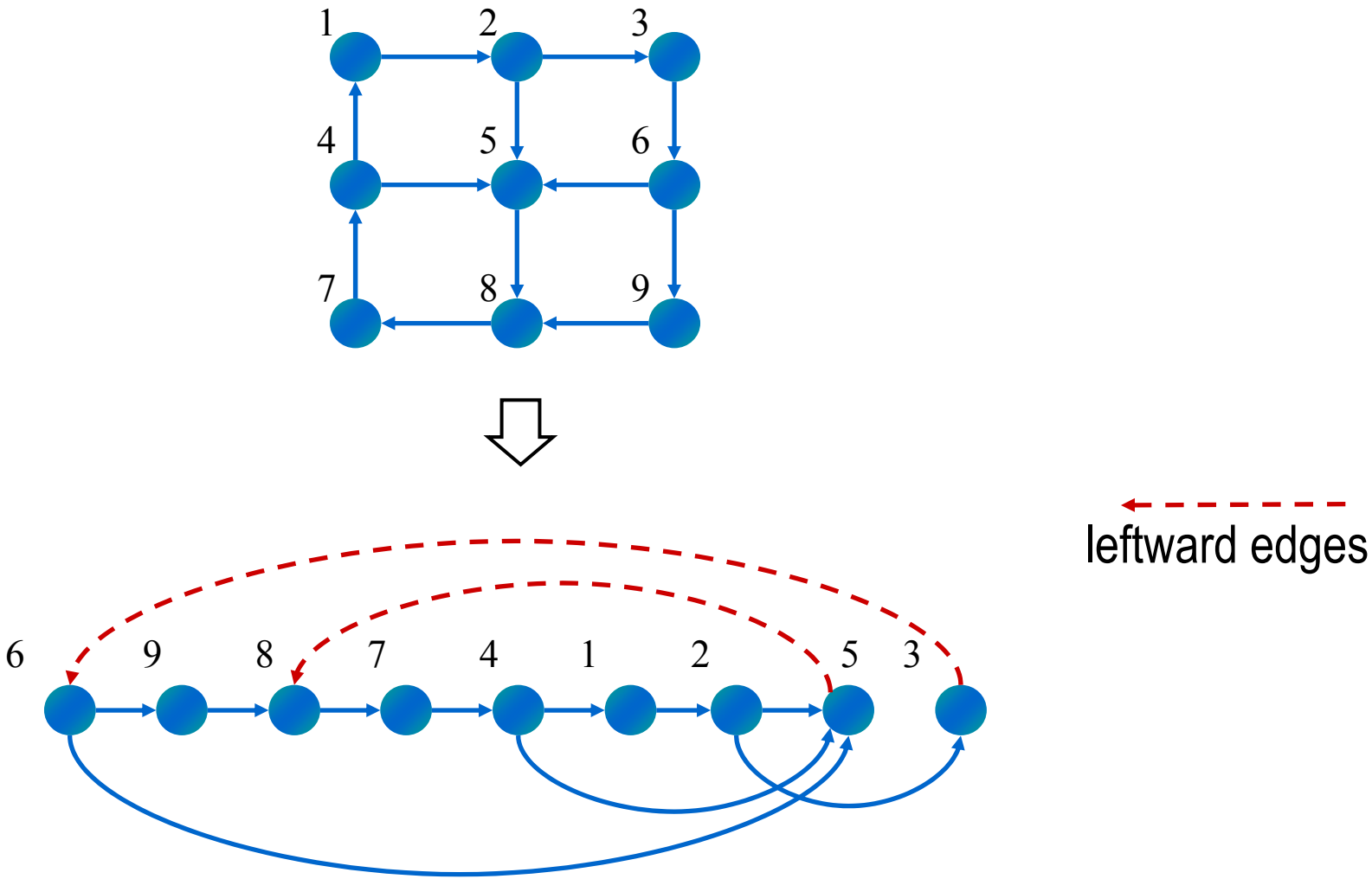
Recasting the problem

- Lets look at the problem from a different perspective
  - let us choose a permutation $\pi = (v_1, v_2, \ldots, v_n)$ of the nodes of G
  - any edge (v_i,v_j) with i > j is said to be a *leftward edge* (with respect to π)



 copyright ©2026 m.patrignani

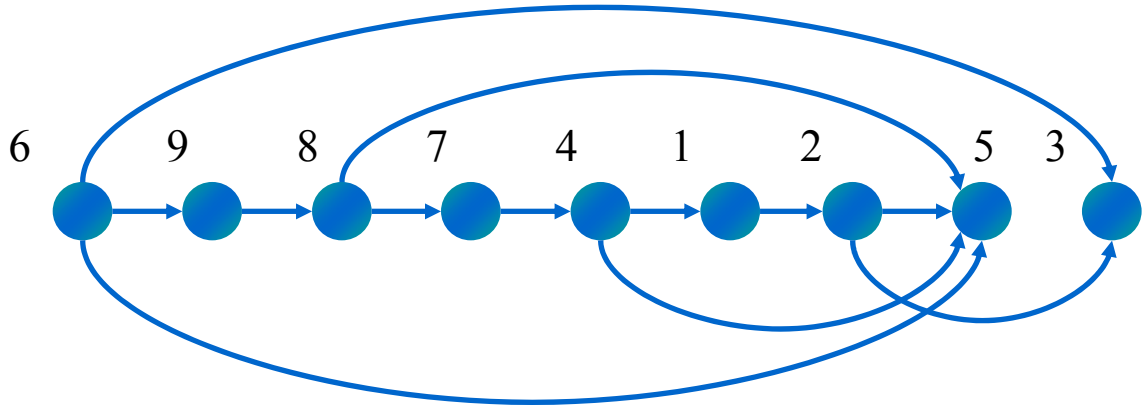
Example



 copyright ©2026 m.patrignani

Topological sort

- A topological sort of a directed graph is a linear ordering of its nodes such that for every directed edge (*u*,*v*), *u* comes before *v* in the ordering
- A topological sort is possible if and only if the graph has no directed cycles



 copyright ©2026 m.patrignani

Feedback set and leftward edges

- Given a permutation π of the nodes of G, the set of leftward edges is a feedback set
  - it is also a feedback arc set
- On the other hand, let R be a feedback set
  - if you compute a topological sort of the directed graph obtained by reverting all edges in R, you obtain a permutation of the nodes whose leftward edge set coincides with R
- Hence, the problem of finding a minimum size feedback set is equivalent to the problem of finding a permutation with the minimum number of leftward edges

 copyright ©2026 m.patrignani

A simple heuristic based on leftward edges

- Arbitrarily choose a permutation of the nodes of G and count the number of leftward edges
  - if this number is greater than half the edges of G, we choose the opposite node sequence
- Size of R
  - the number of reverted edges is at most half the total number of the edges of G
  - this heuristic is surely preferable with respect to the previous one whenever: $|E|/2 < |E|-|V|+1$
    - that is, whenever $|E| > 2|V| - 2$

 copyright ©2026 m.patrignani

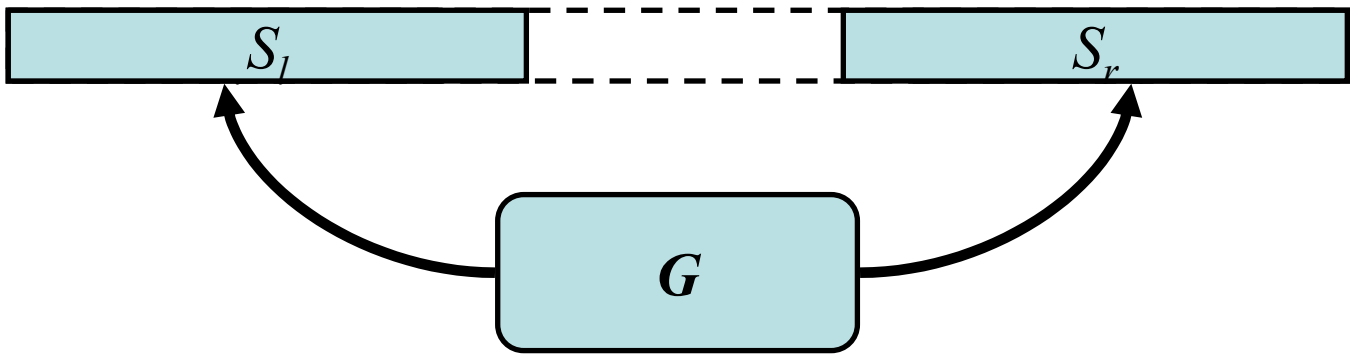
A greedy approach

- A more sophisticated strategy is based on a greedy technique
  - it searches for a sub-optimum solution through a sequence of steps focused on local optima
  - proposed by [Eades *et al.*, 93]

 copyright ©2026 m.patrignani

Greedy cycle removal

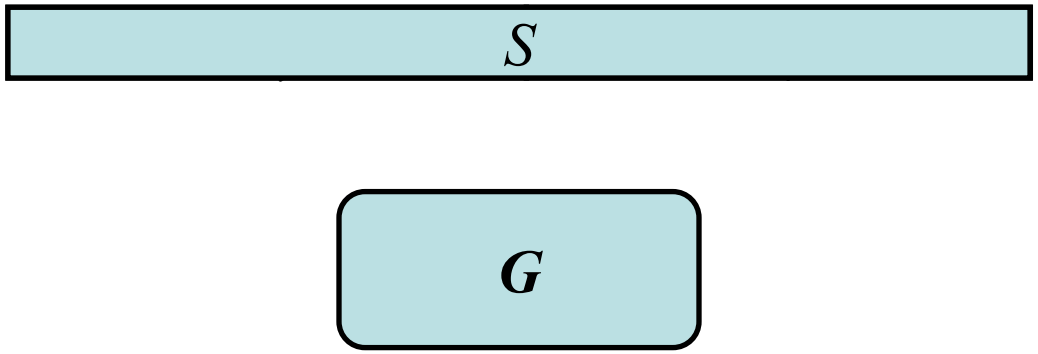
- The algorithm iteratively removes nodes from G and adds them to one of two lists S_L and S_R



 copyright ©2026 m.patrignani

Greedy cycle removal

- The algorithm iteratively removes nodes from G and adds them to one of two lists S_L and S_R
- When all nodes have been removed the sequence S is obtained as a concatenation of the two sequences S_L and S_R



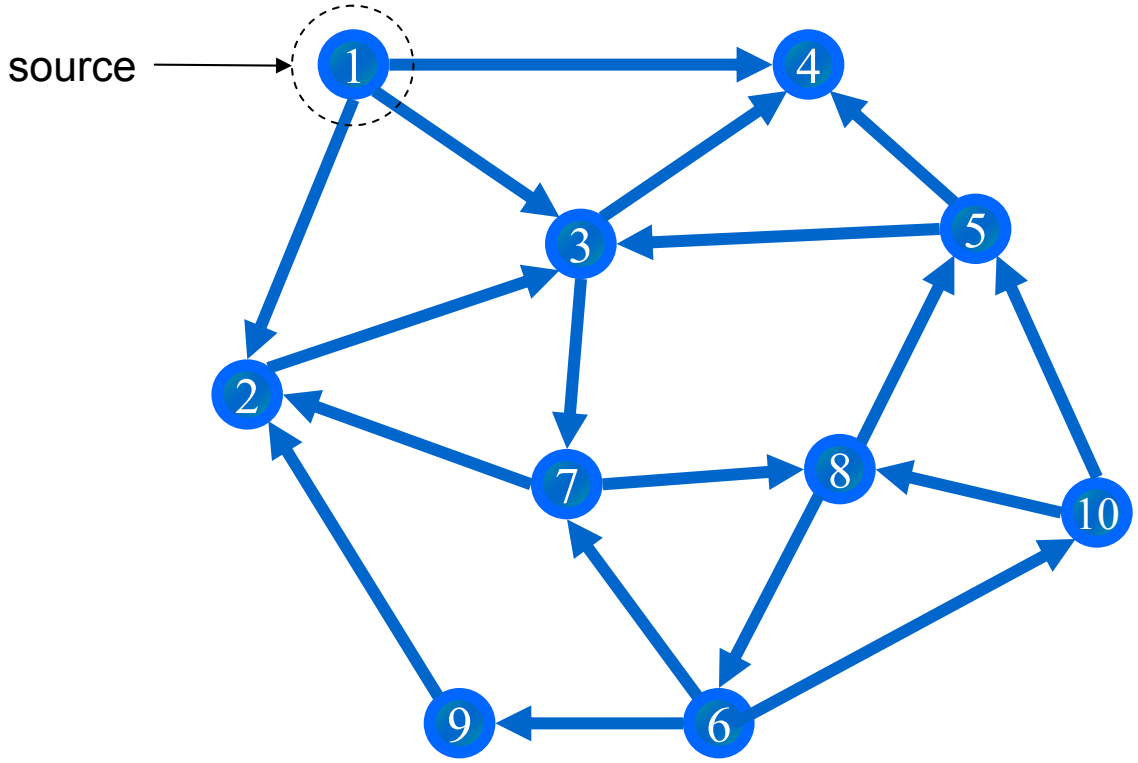
 copyright ©2026 m.patrignani

Greedy choices

- The algorithm chooses in a greedy way the node to be removed from G and adds it to S_L or S_R
  - if there is a source add it at the beginning of S_L
  - if there is a sinks add it at the end of S_R
    - isolated nodes are also considered sinks
  - if there is no sink or source choose a node *u* for which outdeg(*u*)-indeg(*u*) is maximum and add it at the beginning of S_L
    - the number of rightward edges is locally maximized with respect to the leftward edges

 copyright ©2026 m.patrignani

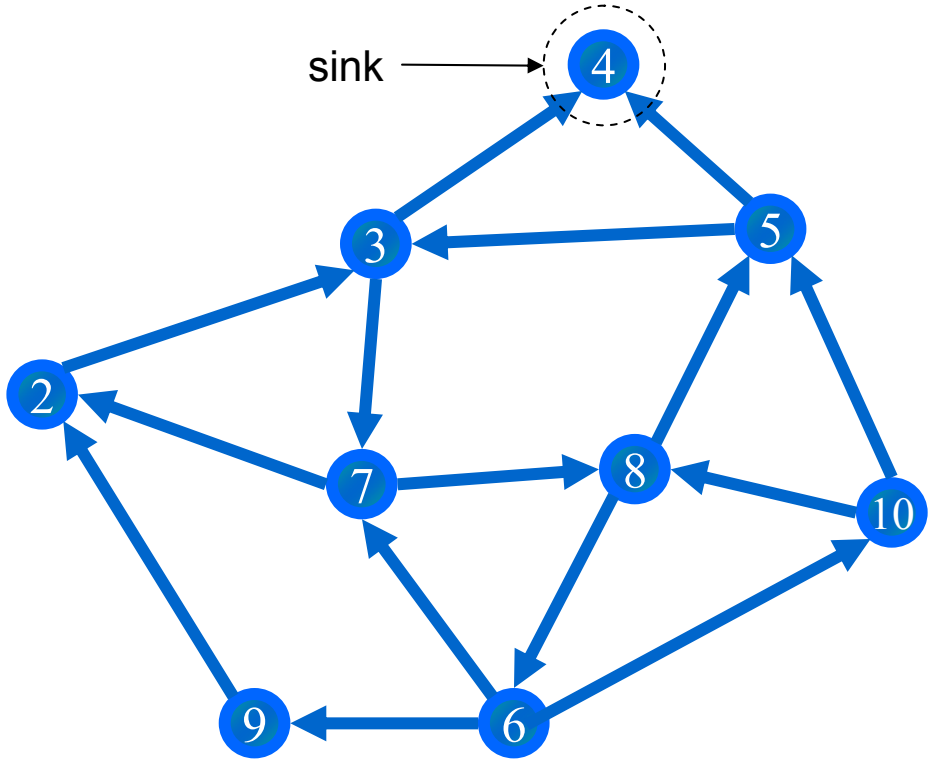
Example



S_L S_R

 copyright ©2026 m.patrignani

Example

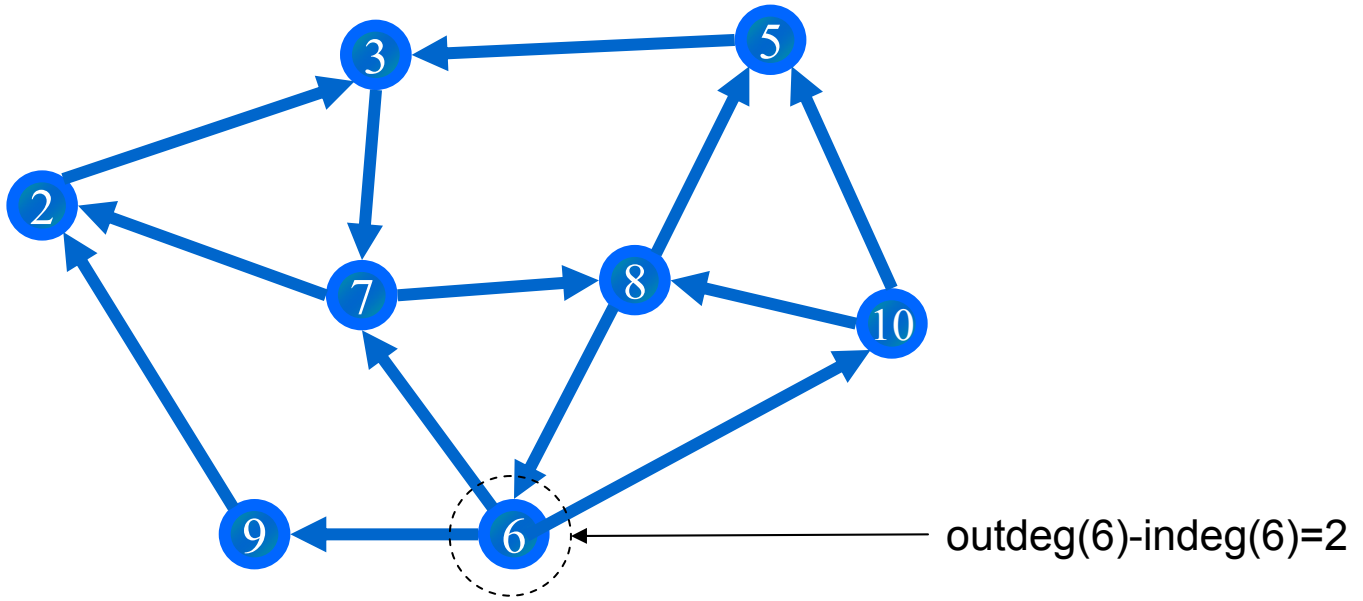


S_L 1

S_R

 copyright ©2026 m.patrignani

Example

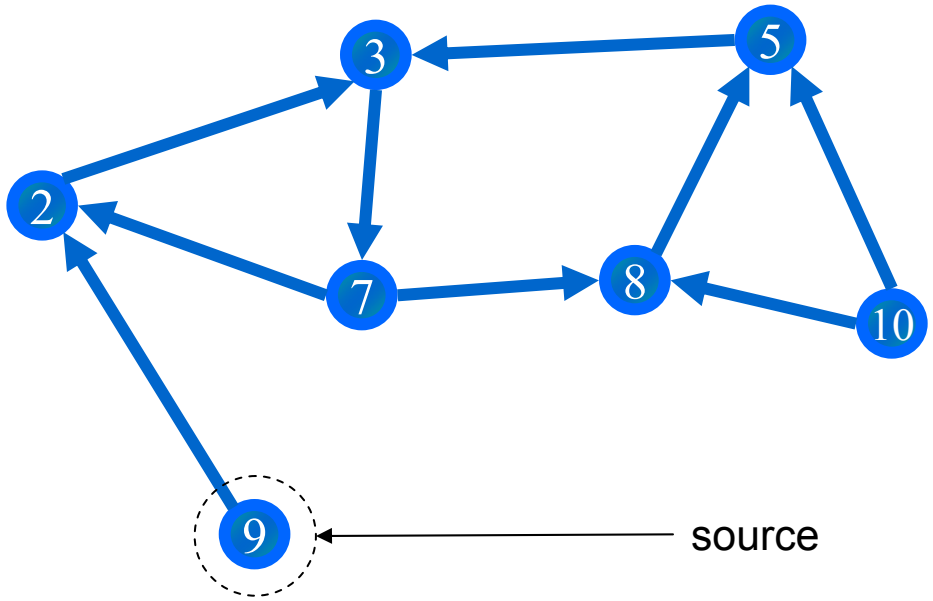


S_L 1

S_R 4

 copyright ©2026 m.patrignani

Example

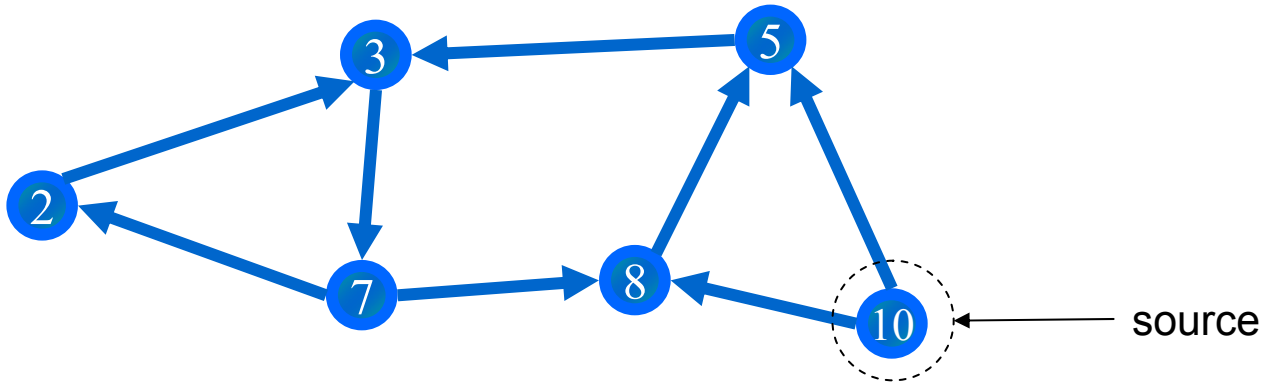


S_L 1 6

S_R 4

 copyright ©2026 m.patrignani

Example

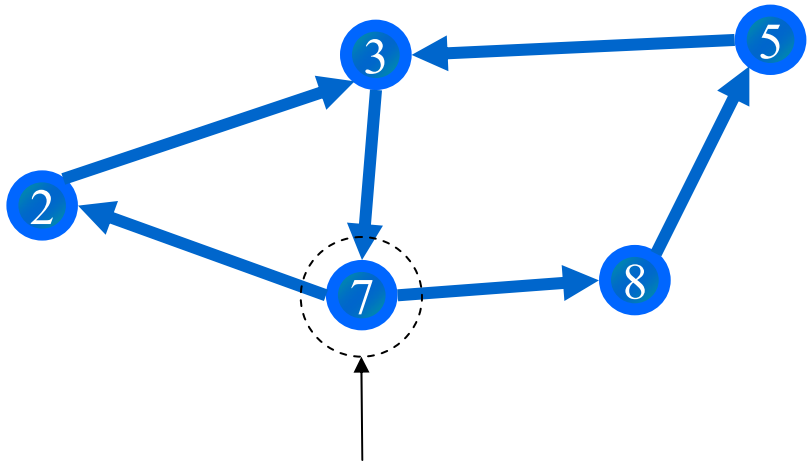


S_L 1 6 9

S_R 4

 copyright ©2026 m.patrignani

Example

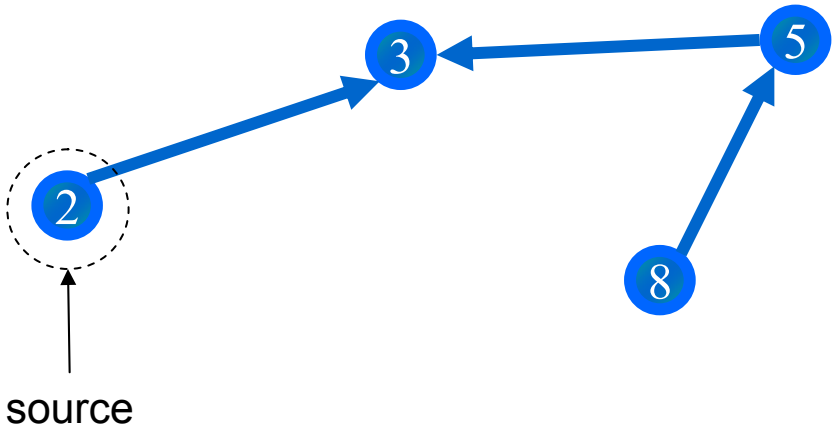


S_L 1 6 9 10

S_R 4

 copyright ©2026 m.patrignani

Example

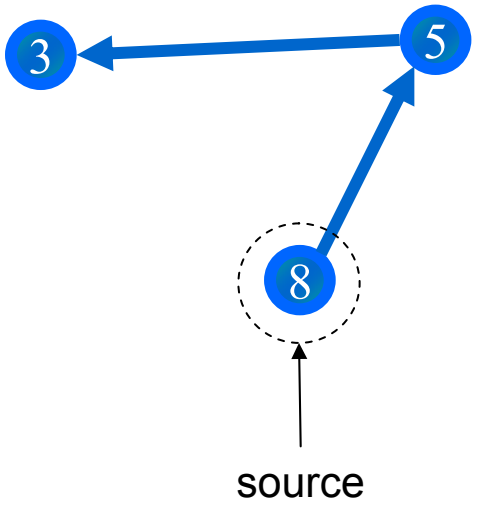


S_L 1 6 9 10 7

S_R 4

 copyright ©2026 m.patrignani

Example

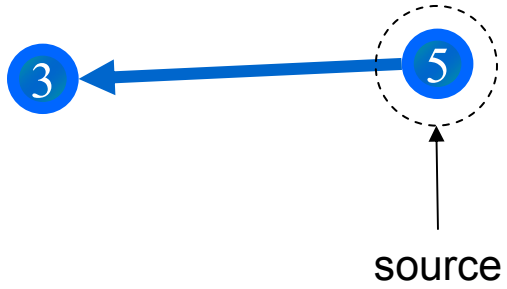


S_L 1 6 9 10 7 2

S_R 4

 copyright ©2026 m.patrignani

Example

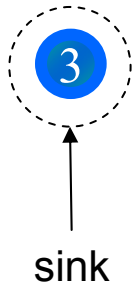


S_L 1 6 9 10 7 2 8

S_R 4

 copyright ©2026 m.patrignani

Example





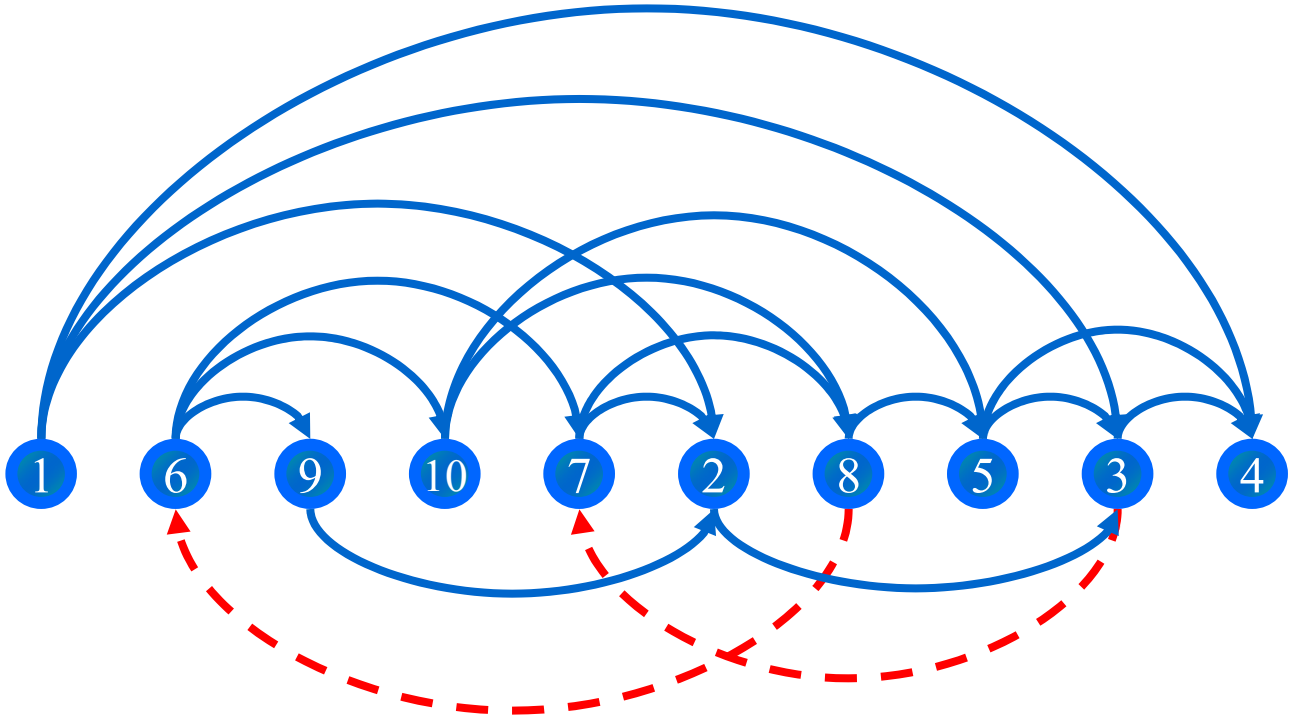
 copyright ©2026 m.patrignani

Example



 copyright ©2026 m.patrignani

Example



S_L 1 6 9 10 7 2 8 5

S_R 3 4

 copyright ©2026 m.patrignani

Algorithm greedy_cycle_removal

```
Greedy_cycle_removal
/* Input: a directed graph G */
/* Output: a node sequence S for G */

    1.  Initialize both S_L and S_R to be empty lists
    2.  While G is not empty do
        (a)  While G contains a sink u or a source v do
                if u is a sink, remove u from G, add u
                as the first element of S_R
                if v is a source, remove v from G, add
                v as the last element of S_L
        (b) if G is not empty then
                choose a node u, such that the
                difference outdeg(u) - indeg(u) is
                maximum, remove u from G and
                add u as the last element of S_L
    3.  Concatenate S_L and S_R to obtain S
```

 copyright ©2026 m.patrignani

Greedy algorithm: properties

- Theorem
  - the greedy algorithm complexity is linear in the number of nodes and edges
  - if the input graph does not contain bi-cycles, the sequence S computed by the greedy algorithm has at most |E|/2 - |V|/6 leftward edges

 copyright ©2026 m.patrignani

Step 2: Level assignment

 copyright ©2026 m.patrignani

Some definitions

- A *layering* of $G=(V,E)$ is a partition of V into subsets $L_1, L_2,\ldots, L_H$, called layers, such that if $(u,v) \in E$, with $u \in L_i$ and $v \in L_j$ then i > j (downwardness)
- The *span* of edge (u,v) with $u \in L_i$ e $v \in L_j$ is i – j
- A layering is *proper* if there is no edges with a span greater than 1
  - not all graphs admit a proper layering
- The *size* of a layering is the minimum area $A = H \times W$
  - H is the number of layers
  - W is the maximum number of nodes sharing the same layer

 copyright ©2026 m.patrignani

Problem statement

Input	A directed acyclic graph G
Output	A proper layering of G
Target	Assign y-coordinates to nodes so that the size of the layering is small

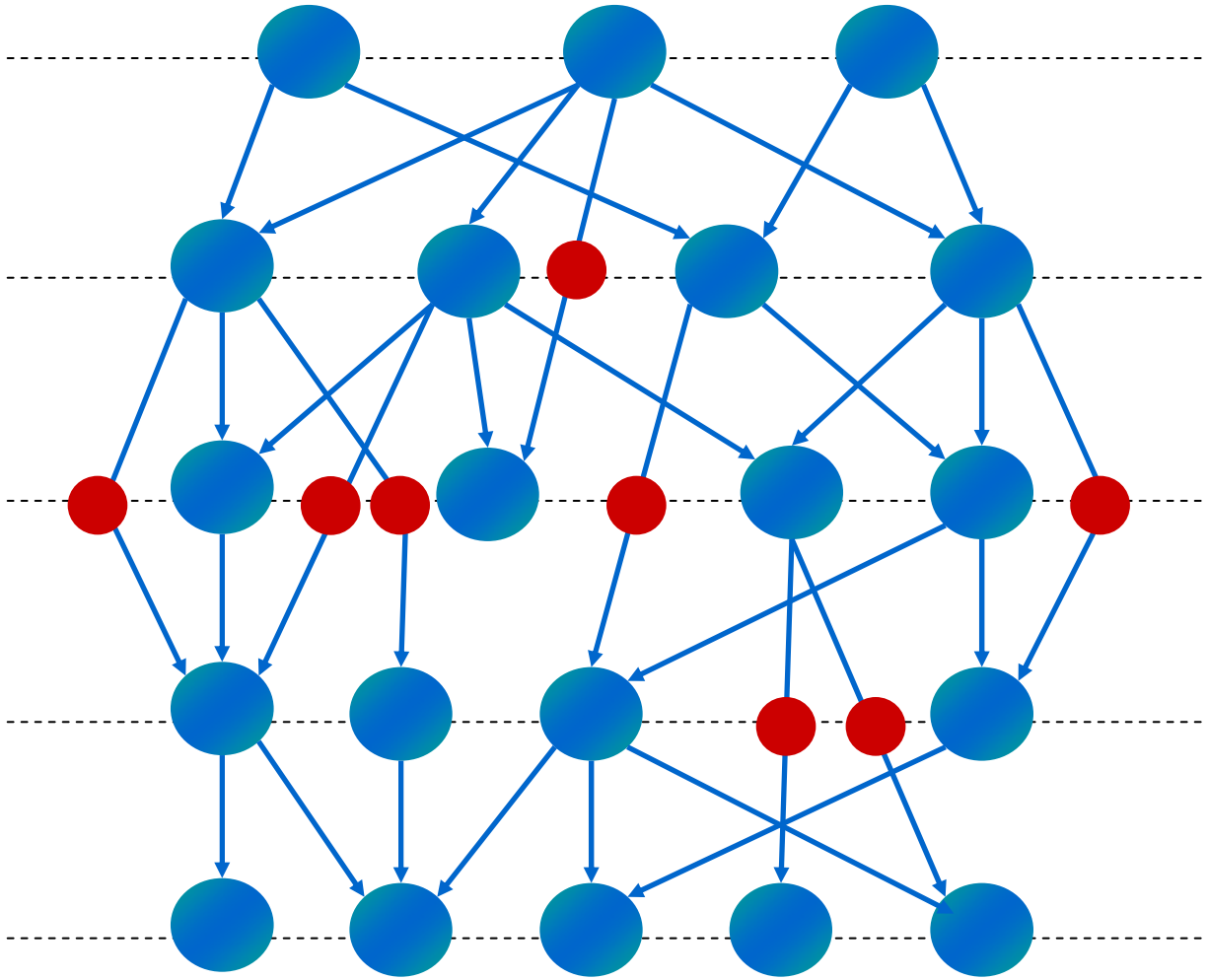
 copyright ©2026 m.patrignani

Proper layering

- It is not always possible to find a proper layering
- Since step 3 needs a proper layering for G, a non-proper layering is transformed into a proper layering as follows:
  - each edge (u,v) with span k > 1 is replaced by a directed path u = v_0,v_1,…,v_k = v, by adding dummy nodes v_1,…,v_{k-1}

 copyright ©2026 m.patrignani

Proper layering



 copyright ©2026 m.patrignani

Optimization criteria

- Reduced area
  - we need both to reduce the number of layers (height reduction), and to uniformly distribute the nodes to the layers (width reduction)
- Reduced edge length
  - each edge should have a small span
- Reduced number of bends
  - edges between adjacent layers are drawn as straight-line segments
  - bends may only occur at dummy nodes
  - reducing the span implies reducing the number of bends

 copyright ©2026 m.patrignani

Minimum size layering problem

- Unfortunately, computing a minimum size layering is NP-complete
  - it is NP-hard deciding whether a directed graph G admits a layering with width W and height H where the width of the dummy vertices is considered to be zero
    - this is proved by reducing Multiprocessor Scheduling [Eades & Sugiyama, 90]
  - it is NP-hard deciding whether a directed graph G admits a layering with width W where the width of the dummy vertices is considered greater than zero
    - [Branke *et al.*, 02]

 copyright ©2026 m.patrignani

Layering heuristics

- Several methods have been proposed to solve the layering problem
  - the Longest-path method
  - the Coffman-Graham method
  - the Gansner et al. method
  - …
- They have different properties with respect to different aesthetic requirements
  - for time constraints we will only see the Longest-path method and the Coffman-Graham method

 copyright ©2026 m.patrignani

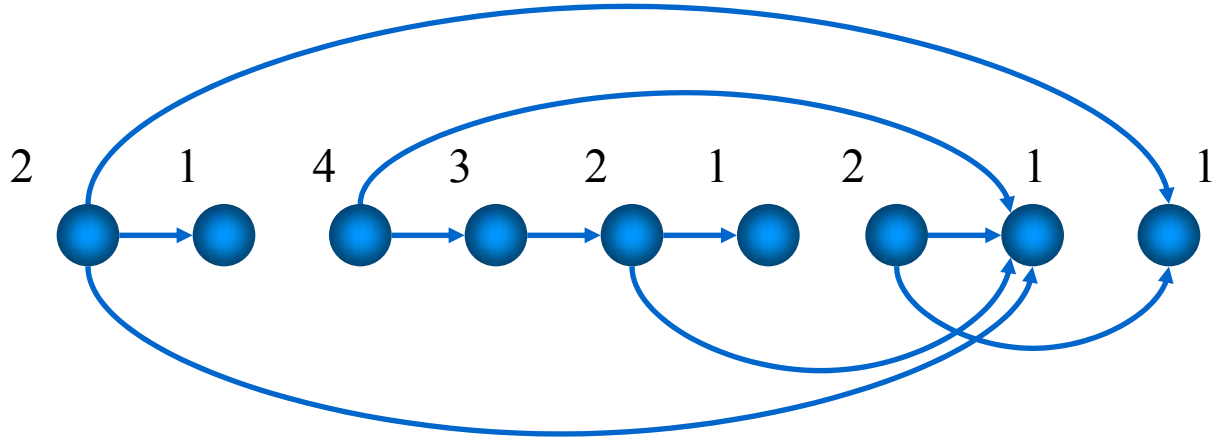
Longest-path method

- Strategy
  - for each node compute the longest path towards each sink
  - for each node take the worst of such lengths
  - assign each node to the level corresponding to this length
- This corresponds to computing what it is called a “topological ordering” of the graph

 copyright ©2026 m.patrignani

Topological orderings

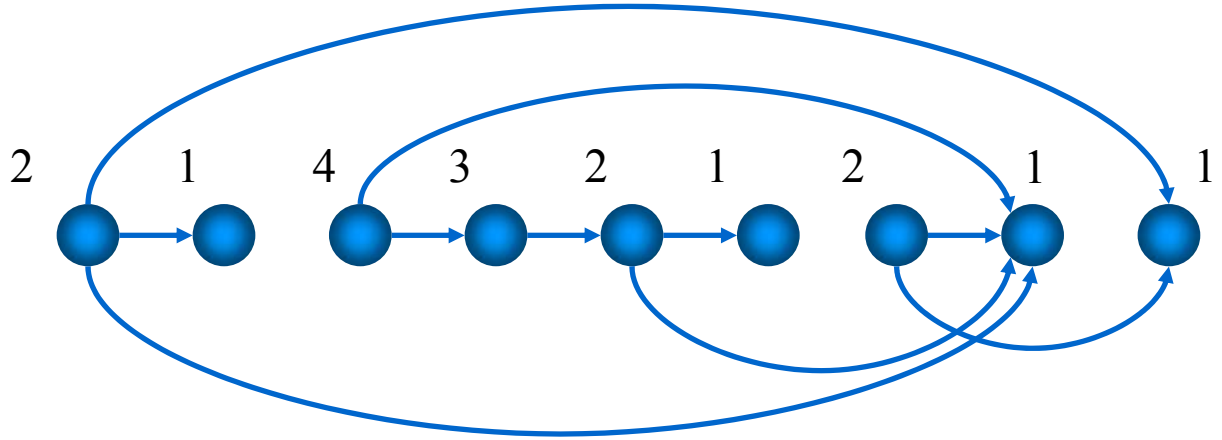
- Let G = (V, E) be a directed acyclic graph
- A *topological ordering* T(G) is an assignment of integer values T(v) to each node v such that for every directed edge (u, v) $\in$ E, we have that T(v) < T(u)



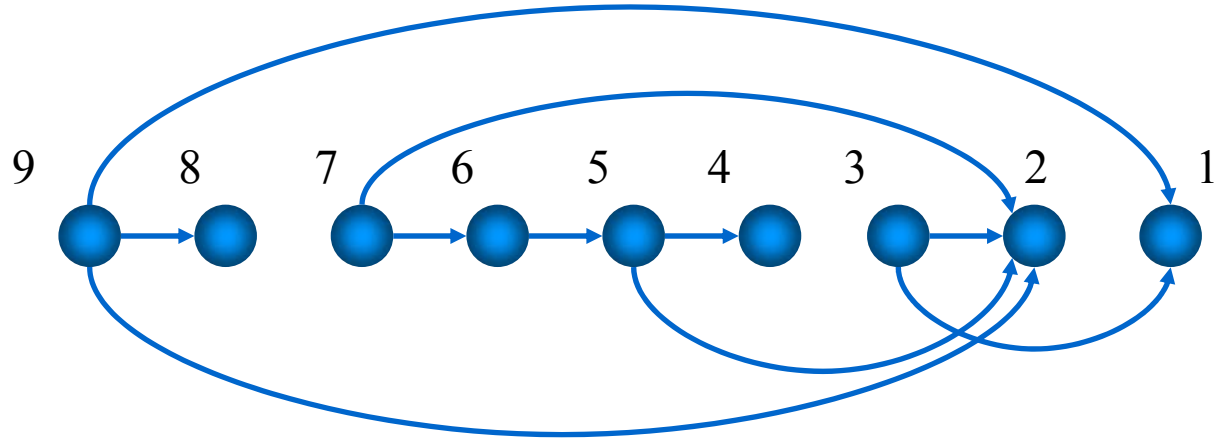
 copyright ©2026 m.patrignani

Topological orderings Vs sorts

- Topological ordering: numbers can be repeated



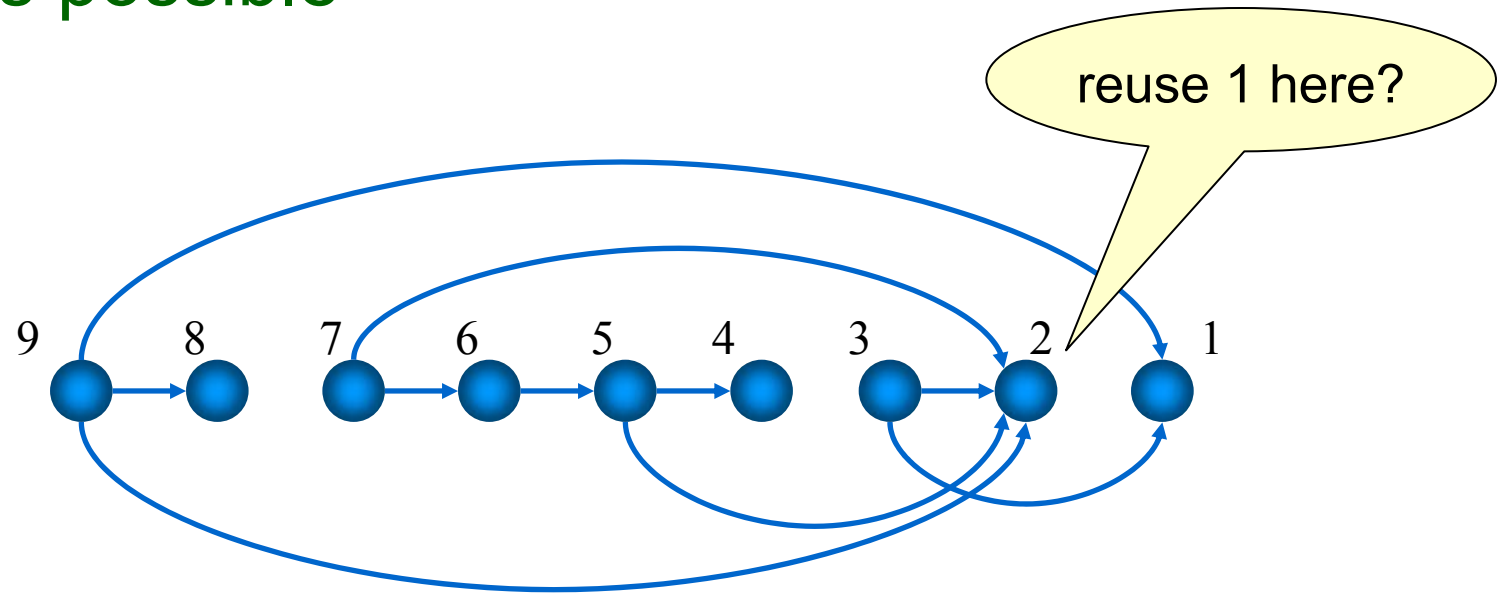
- Topological sort: numbers are unique



 copyright ©2026 m.patrignani

Topological orderings Vs sorts

- A topological sort is a topological ordering
  - but not vice versa
- A topological sort is a topological ordering that wastes a lot of numbers!
  - we would like to have topological orderings that use as few numbers as possible



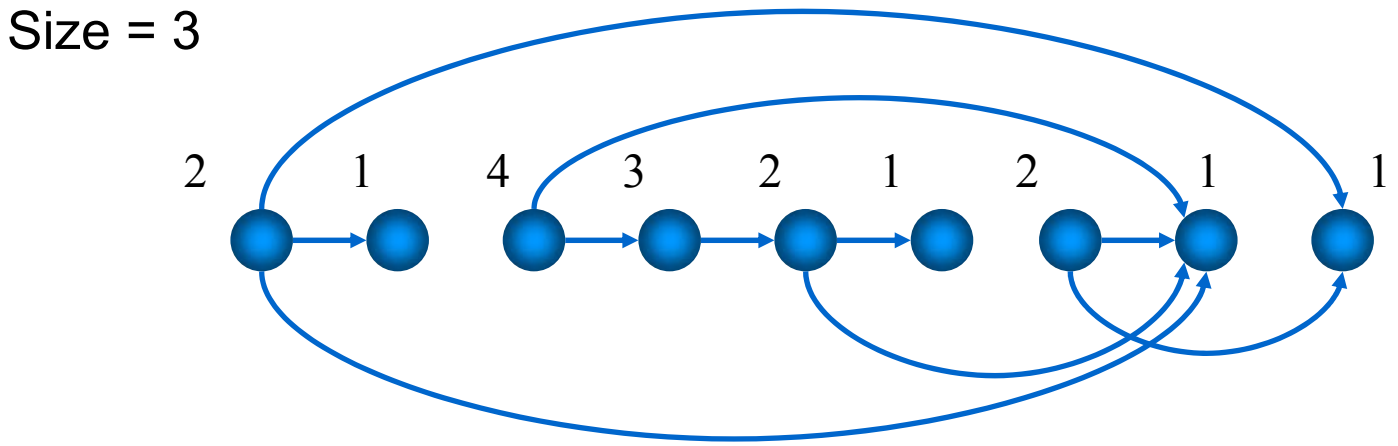
 copyright ©2026 m.patrignani

Size of a topological ordering

- The *size* of the topological ordering is

$$\max_{v \in V}\left(T(v)\right) - \min_{u \in V}\left(T(u)\right)$$

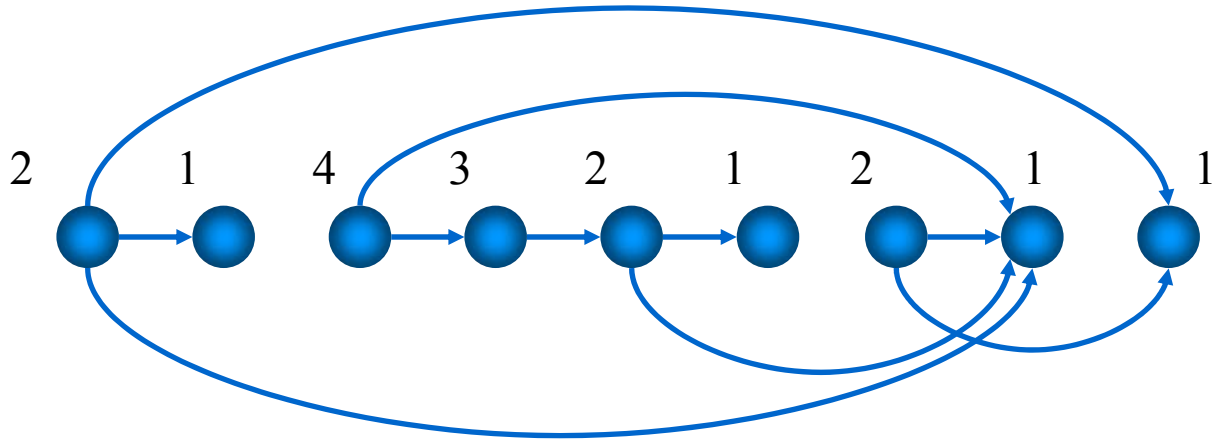
- An *optimal topological ordering* T*(G) is a topological ordering with the smallest size



 copyright ©2026 m.patrignani

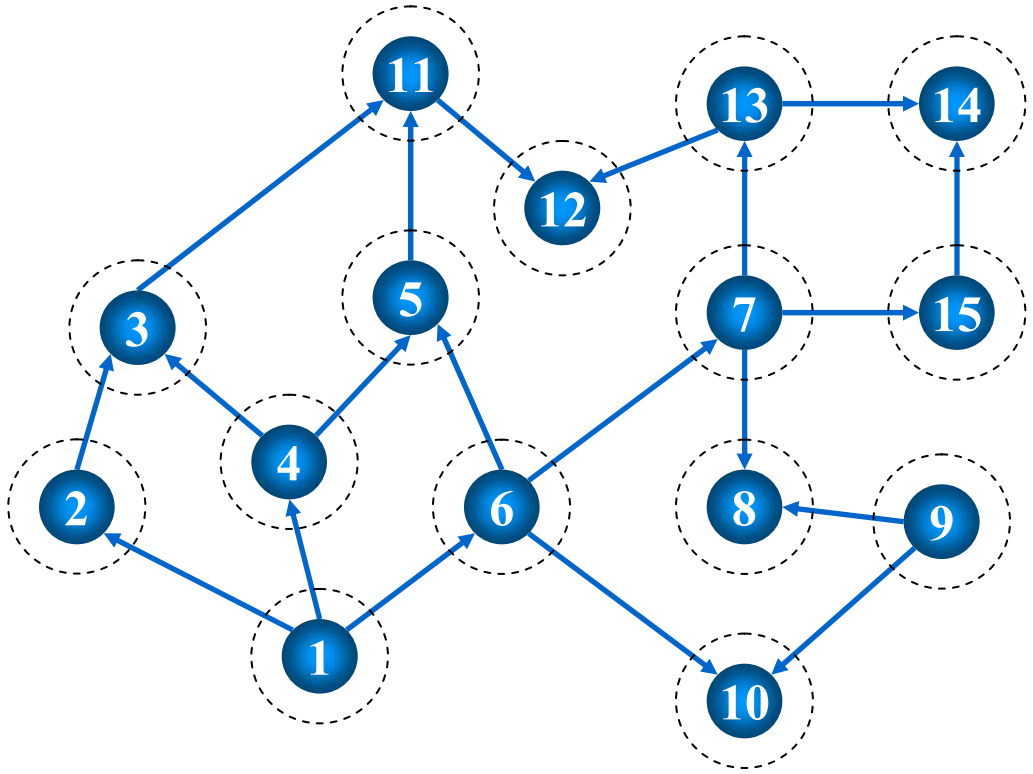
Computing topological orderings

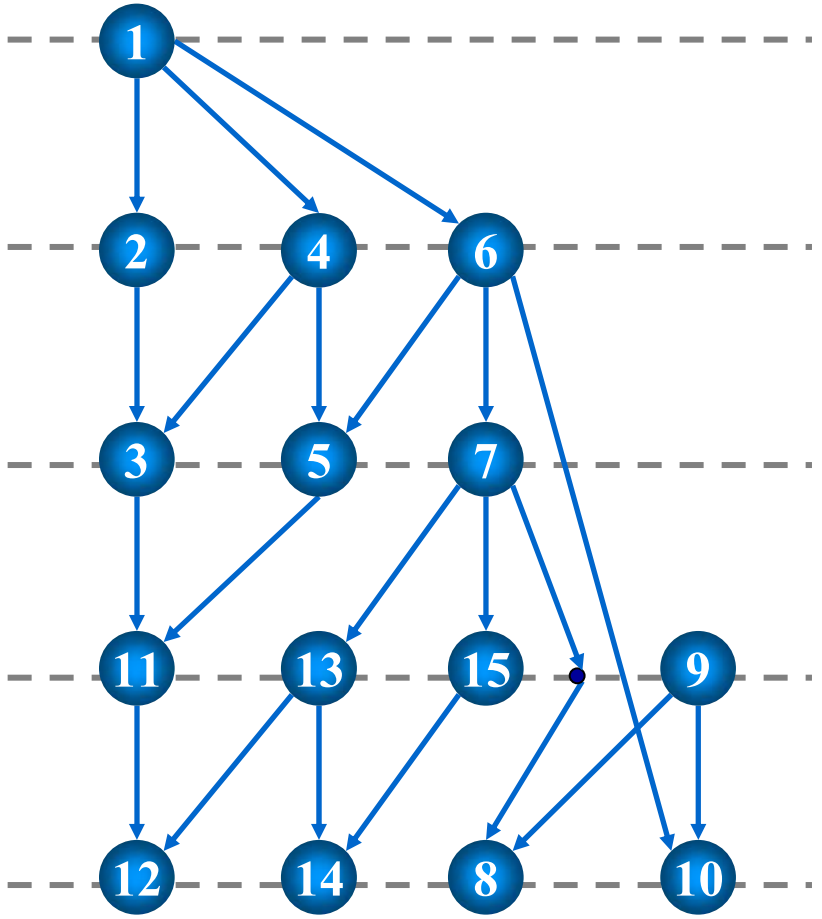
- Computing an optimal topological ordering of a DAG in linear time is simple
  - assign every sink node the number s=1
  - remove sink nodes and their edges from the graph
  - repeat with the number s+1 until there is no node left



 copyright ©2026 m.patrignani

Example of layer assignment





 copyright ©2026 m.patrignani

Complexity and properties

- The complexity of the Longest-path algorithm is that of the computation of the longest paths for all the nodes of the graph
  - that is, O(n+m) since the graph is acyclic
- The algorithm finds a layering with the minimum number of layers
  - the height of the layering is minimum
- The width of the layering is not guaranteed to be the minimum width
  - on the contrary, the algorithm usually produces layerings in which nodes are not uniformly distributed among layers

 copyright ©2026 m.patrignani

Multiprocessor scheduling

- Problem “Multiprocessor scheduling with precedence constrains” is as follows

Input	A set of tasks
	each requiring the same cpu time
	A set of precedence constraints
	a task t_i can not be executed if
	another task t_j is not terminated
	W processors and a time H
Output	An assignment of tasks to processors
Target	All tasks are executed in at most H time

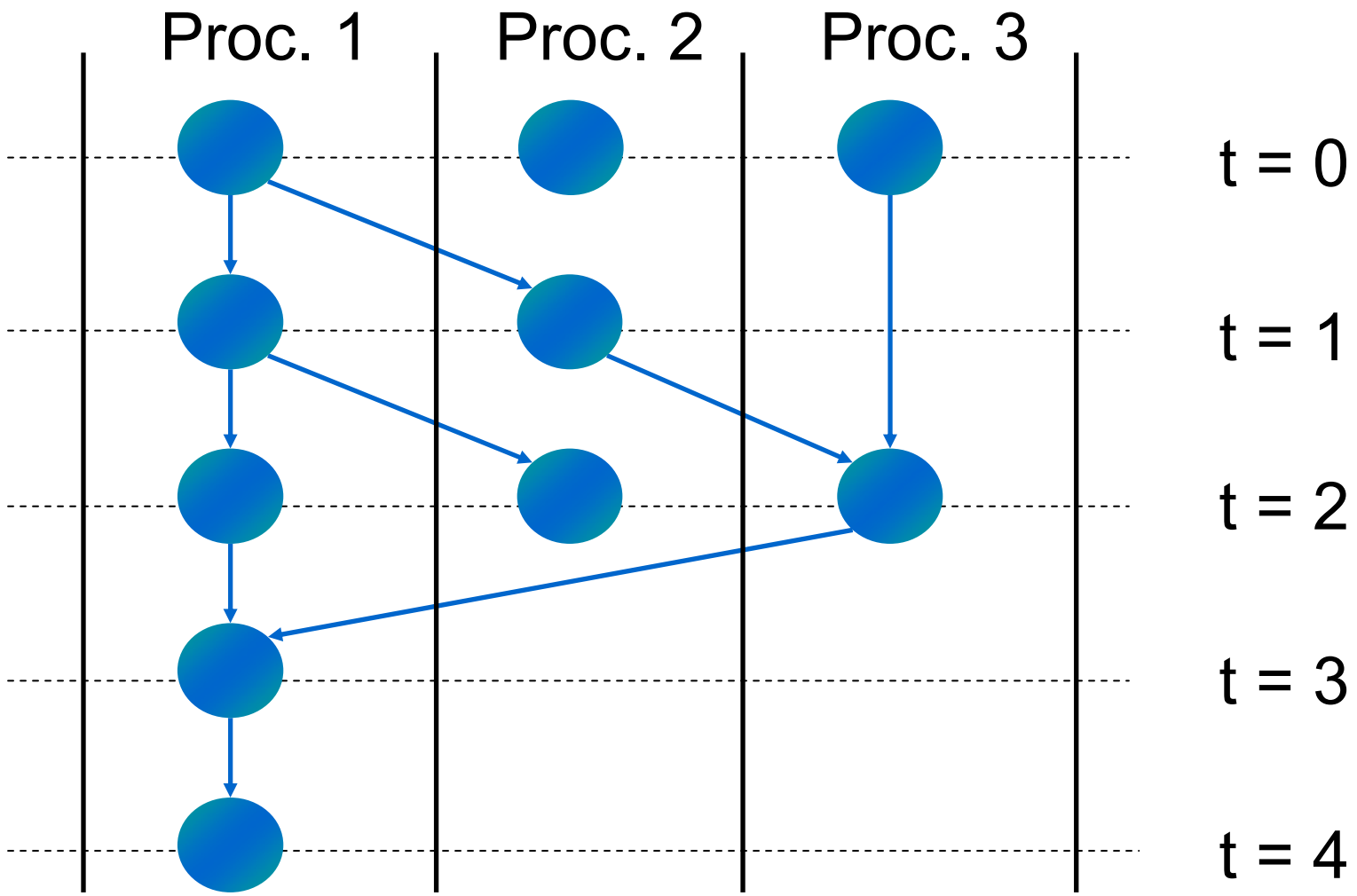
 copyright ©2026 m.patrignani

Problem equivalence

- Each task corresponds to a node of a DAG G
- Each precedence constraint is a direct edge between the corresponding tasks
- We may assume that G is an acyclic DAG
  - if G were not acyclic the instance would be trivially a false instance
- Each time between 1 and H corresponds to a layer
- At each moment at most W tasks may be computed
  - we have W processors only
- The tasks can be executed in H time if and only if G admits a layering with width W and height H

 copyright ©2026 m.patrignani

Equivalence example



 copyright ©2026 m.patrignani

Coffman-Graham heuristic

- Described in [Coffman & Graham, ‘72]
- Inspired by the connection with the multiprocessor scheduling problem
- The input is a width W and a DAG G without transitive edges
- The output is a layering with width at most W and (hopefully) reduced height H
- Dummy nodes introduced by transitive edges are not considered in W
  - they don’t host labels, and may have less inpact on the width of the final drawing

 copyright ©2026 m.patrignani

Coffman-Graham layering algorithm

- The Coffman-Graham layering algorithm has two phases
  - in the first phase vertices are suitably ordered and numered 1, 2, ..., n
    - higher numbers correspond to vertices that is preferable to put in the lower layers of the diagram
  - in the second phase vertices are assigned to layers

 copyright ©2026 m.patrignani

Problem addressed in the first phase

Input	A directed graph $G=(V,E)$ and an integer k
Output	A bijection π: $V \rightarrow \{1, 2, ..., \|V\|\}$ such that for all $(u,v) \in E$ you have $\pi(u) < \pi(v)$
Target	For all $(u,v) \in E$, $\pi(v) - \pi(u) < k$

- This problem, called *directed bandwidth* is shown to be NP-hard in [Garey et al 78]
- Hence, the Coffman-Graham algorithm uses a heuristic

 copyright ©2026 m.patrignani

Definitions

- We define a lexicographic order among sets
- Given two (finite) sets of positive integers S and T we say that S < T if
  - S is empty and T is not empty, or
  - max(S) < max(T), or
  - max(S) = max(T) and S – {max(S)} < T – {max(T)}
- Examples

$$\{\} < \{5, 3, 1\}$$
$$\{6, 4, 3, 2\} < \{7, 5, 4, 2, 1\}$$
$$\{5, 3, 1\} < \{5, 4, 2\}$$
$$\{7, 2\} < \{7, 2, 1\}$$

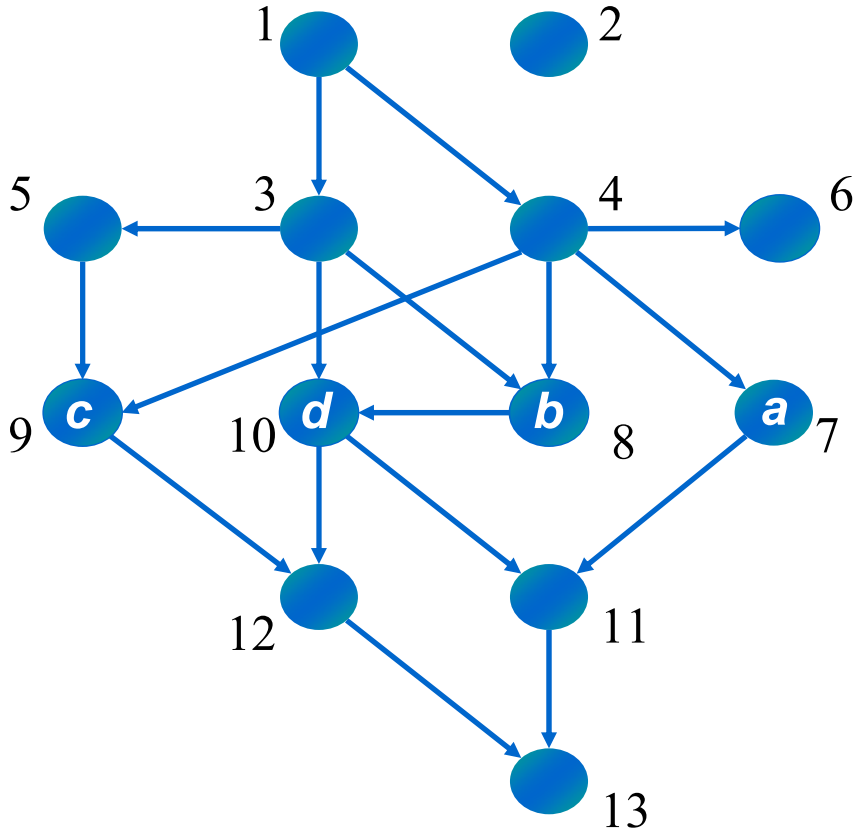
 copyright ©2026 m.patrignani

Phase 1 of Coffman-Graham

- First, the nodes of G are ordered by assigning a label $\pi(u)$ to each node u
  - you start by assigning $\pi(u)=1$ to any source u
  - suppose you assigned labels 1, 2, …, k-1
  - the label k is assigned to a node v if
    - v has no label yet
    - all edges entering v exit from labeled nodes
    - among all nodes satisfying the two conditions above choose the v such that the set of labels of immediate predecessors of v is the first in the lexicographic order

copyright ©2026 m.patrignani

Phase 1 example



- Observe that
  - *a* is labeled before *b* since {4} < {4, 3}
  - *c* is labeled before *d* since {5, 4} < {8, 3}

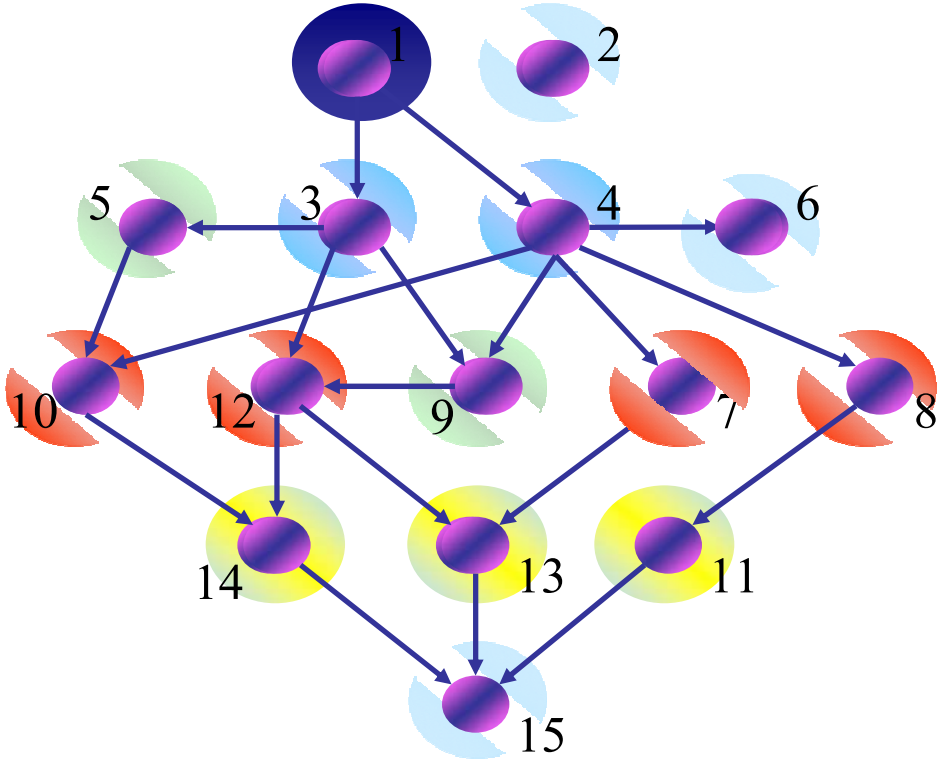
 copyright ©2026 m.patrignani

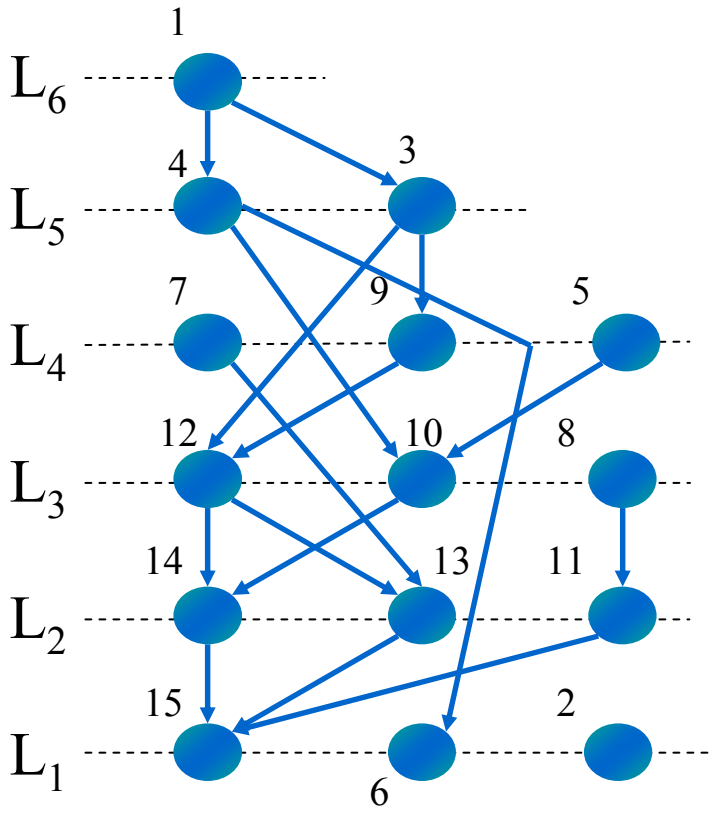
Phase 2 of Coffman-Graham

- Each layer is assigned a maximum of W nodes
- You start from layer L_1 at the bottom
- In order to fill layer L_k
  - choose a node v that is not assigned to any layer and such that all nodes u with $(v, u) \in E$ are already assigned to lower layers
    - sinks and isolated nodes are assigned first
  - if there is more than one candidate node choose the one with the highest label
  - if there is no candidate node or if the layer has W nodes procede to the next layer

 copyright ©2026 m.patrignani

Phase 2 example





W=3

 copyright ©2026 m.patrignani

Algorithm Coffman_Graham

```
Coffman_Graham
/* Input:   a directed graph G without
            transitive edges and a positive
            integer W */
/* Output: a layering with width at most W */

   /* Phase 1: node ordering */
   1. Initially all nodes have no label
   2. for i=1 to |V| do
      (a)   Choose a non-labeled node v such
            that the set {π(u):(u,v)∈E} is
            minimized
      (b)   Set π(v) = i
```

 copyright ©2026 m.patrignani

Algorithm Coffman_Graham

```
/* Phase 2: layering */
3. Set k=1; L_1=∅; U=∅ (U = already placed nodes)
4. While U≠V do
   (a) Choose u ∈ V-U such that:
         (i) {v:(u,v)∈ E} ⊆ U
         (ii) π(u) is maximized
   (b) If |L_k| < W and for each edge (u,w)
         you have that w ∉ L_k
       Then add u to L_k
       Else k = k+1, L_k={u}
   (c) Add u to U
```

 copyright ©2026 m.patrignani

Properties of Coffman-Graham

- By construction the width is at most W
- In general, the height h of the layering is not the minimum height h_{min}
- It can be proved that

$$h \leq (2 - 2 / W)\ h_{min}$$

  - hence, if W = 2 we have

$$h = h_{min}$$

 copyright ©2026 m.patrignani

Bibliography and further readings

- [Branke *et al.*, 02] J. Branke, S. Leppert, M. Middendorf, and P. Eades. “Width-restricted layering of acyclic digraphs with consideration of dummy nodes” IPL, 2002
- [Coffman & Graham, 72] Edward G. Coffman and Ronald Graham, “Optimal scheduling for two-processor systems”, Acta Informatica 1, 1972
- [Di Battista et al 1998] Giuseppe Di Battista, Renato Lillo, Fabio Vernacotola. "Ptolomaeus: The Web Cartographer". Graph Drawing '98. Pages 444-445. 1998
- [Eades & Sugiyama, 90] P. Eades and K. Sugiyama. “How to draw a directed graph” Journal of Information Processing, 1990
- [Eades *et al.*, 93] Peter Eades, Xuemin Lin, W. F. Smyth “A Fast and Effective Heuristic for the Feedback Arc Set Problem”, Inf. Process. Lett., 1993
- [Forster et al 2002] Michael Forster, Andreas Pick, Marcus Raitner, Franz Brandenburg. "The system architecture of the BioPath system". Silico Biology 2(3):415-26. 2002
- [Garey et al 78] M.R. Garey, R.L. Graham, D.S. Johnson, D.E. Knuth. ”Complexity results for bandwidth minimization”, SIAM J. of Appl. Math. 1978
- [Grinberg & Dambit, 66] E.J. Grinberg and J.J. Dambit. Latv. Mat. E., 2:65–70, 1966
- [Karp 72] Richard M. Karp. “Reducibility among combinatorial problems”. Complexity of Computer Computations, PLNM, 1972
- [Sugiyama *et al*., 81] Kozo Sugiyama, Shojiro Tagawa, Mitsuhiko Toda, “Methods for Visual Understanding of Hierarchical System Structures” IEEE Trans. on Systems, Man, and Cybernetics,1981

 copyright ©2026 m.patrignani